

KI@Informatik11

Was, wozu, wie, womit unterrichten?



Arbeitsheft zur Lehrerfortbildung

Konzeption des Arbeitsheftes:

Das Arbeitsheft wird in den Lehrerfortbildungen „KI@Informatik 11 – was, wozu, wie, womit unterrichten“ der Didaktik der Informatik der Universität Passau zum Thema „Künstliche Intelligenz“ in der 11. Jahrgangsstufe verwendet.

Dr. Wolfgang Pfeffer

Dominicus-von-Linprun-Gymnasium Viechtach

E-Mail: schule@pfeffer-wolfgang.de

Tobias Fuchs

Universität Passau

E-Mail: fuchs_unipa@outlook.de

Das Material zu den Arbeitsaufträgen findet sich im Mebis-Kurs „Material zu den KI-Fortbildungen der Universität Passau“ mit der ID-Nummer 1324361. Der Einschreibeschlüssel ist KI_FB_UP.



Die enthaltenen Bilder sind (falls nicht anders angegeben) der Plattform www.pixabay.com entnommen und wurden teilweise weiter bearbeitet. Diese können unter der dort aufgeführten [Inhaltslizenz](#) kostenlos genutzt werden, wobei kein Bildnachweis nötig ist.



Inhaltsverzeichnis

1. Entscheidungsbäume	4
1.1 Kompetenzen laut Lehrplan	4
1.2 Trainings- und Testdaten	4
1.3 Erstellen des Entscheidungsbaums anhand der Trainingsdaten	6
1.4 Testen des Entscheidungsbaumes	12
1.5 Entscheidungsbäume in Orange	13
2. Der k-nächste-Nachbarn-Algorithmus	19
2.1 Kompetenzen laut Lehrplan	19
2.2 "Was bin ich? Stadt, Land, Tier-Edition" - KNN-Algorithmus und Scratch	19
2.3 Klassifikation mit dem k -nächste-Nachbarn-Algorithmus	20
2.4 Der Lernprozess des KNN-Algorithmus	23
2.4.1 Phase 0: Vorbereitung	24
2.4.2 Phase 1: Training	24
2.4.3 Bestimmung eines passenden Werts für k	25
2.4.4 Phase 2: Testen	28
2.5 Mögliche Einflussfaktoren	30
2.6 Regression als weitere Anwendung des KNN-Algorithmus	32
3. Künstliches Neuron – Das Perzeptron	34
3.1 Kompetenzerwartungen laut Lehrplan	34
3.2 Aufbau und Funktionsweise eines Perzeptrons	34
3.3 Wie lernt ein Perzeptron?	36
3.4 Geometrische Aspekte des Perzeptrons	40
3.5 Grenzen des Perzeptrons	41
3.6 Implementierung des Perzeptrons	42
3.6.1 Implementierung in JAVA	42
3.6.2 Implementierung in Python	45
3.7 Testen des Modells	46
3.8 Vom Perzeptron zum neuronalen Netz	47
4. Chancen und Risiken von Künstlicher Intelligenz für Individuum und Gesellschaft	48
4.1 Chancen und Risiken und deren Bedeutung	48
4.2 Die Wahl des Qualitätsmaßes und dessen Auswirkungen	49
4.3 Überblick über Aspekte zur Beurteilung eines KI-Systems	50
5. Paradigmenwechsel im Problemlöseprozess mit Algorithmen	52

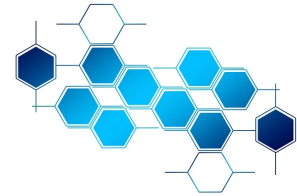


1. Entscheidungsbäume

1.1 Kompetenzen laut Lehrplan

Kompetenzerwartungen Informatik 11 (NTG / *spät beginnend*):

- erläutern die **Funktionsweise / Idee** eines ausgewählten Algorithmus maschinellen Lernens (k -nächste-Nachbarn- oder Entscheidungsbaum-Algorithmus) **allgemein und** an konkreten Beispielen.
- analysieren den Einfluss von Trainingsdaten und Parametern auf die Zuverlässigkeit der Ergebnisse eines Verfahrens maschinellen Lernens, ggf. unter Verwendung eines geeigneten Werkzeugs.



1.2 Trainings- und Testdaten¹



Arbeitsauftrag 1: Wie würden Sie entscheiden?

Sie bekommen einen Datensatz mit 14 Fischen, von denen Sie jeweils wissen, ob diese friedlich oder feindselig sind. Versuchen Sie anhand dieses Datensatzes für zwei neue Fische derselben Spezies zu entscheiden, ob Sie diese ohne Bedenken in Ihr Aquarium geben können!

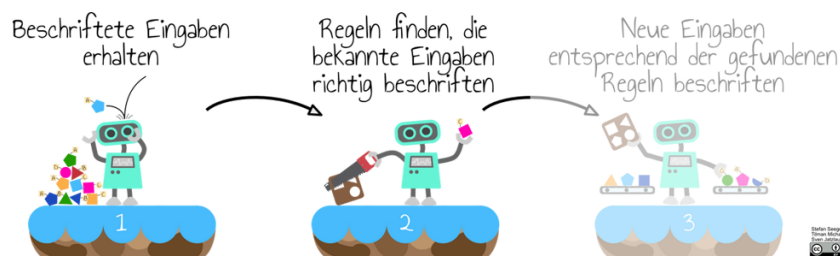


¹Ein ähnliches Vorgehen findet sich bei „AIUnplugged“ (www.aiunplugged.org).



Zentrale Idee (Trainingsdaten vs. Testdaten)

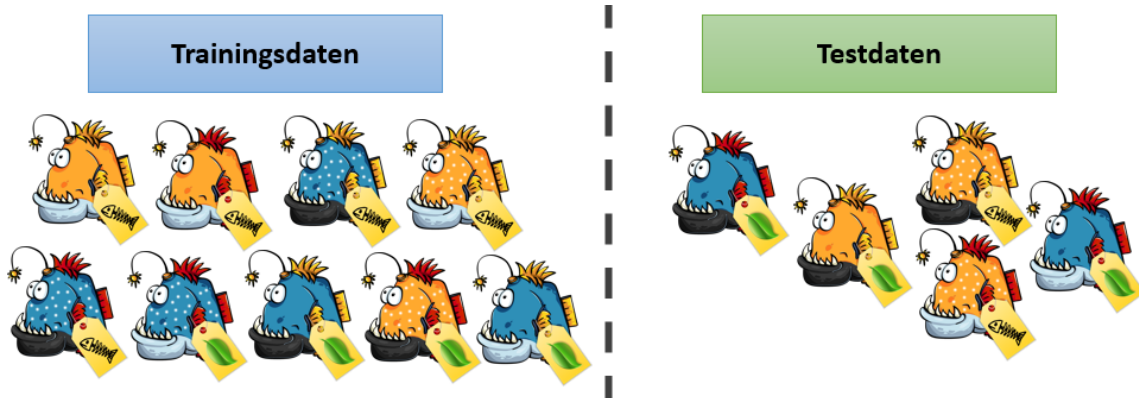
Gemäß dem Verfahren des überwachten Lernens wird ein Modell (hier ein Entscheidungsbaum) ausgehend von gelabelten Trainingsdaten erstellt:



Aber wie gut ist dieser Entscheidungsbaum? Wie kann die „Vorhersage-Qualität“ des Entscheidungsbaumes überprüft werden?

Die „Güte“ des Baumes kann mit Hilfe von Daten getestet werden, bei denen das Label bekannt ist. Hierzu unterteilt man die verfügbare Datenmenge vorab in **Trainingsdaten**, ggf. **Validierungsdaten** (später) und **Testdaten**. Die Trainingsdaten werden zum Erstellen des Baumes verwendet, die Testdaten im Anschluss, um die Güte bzw. „Vorhersage-Qualität“ des Baumes beurteilen zu können.

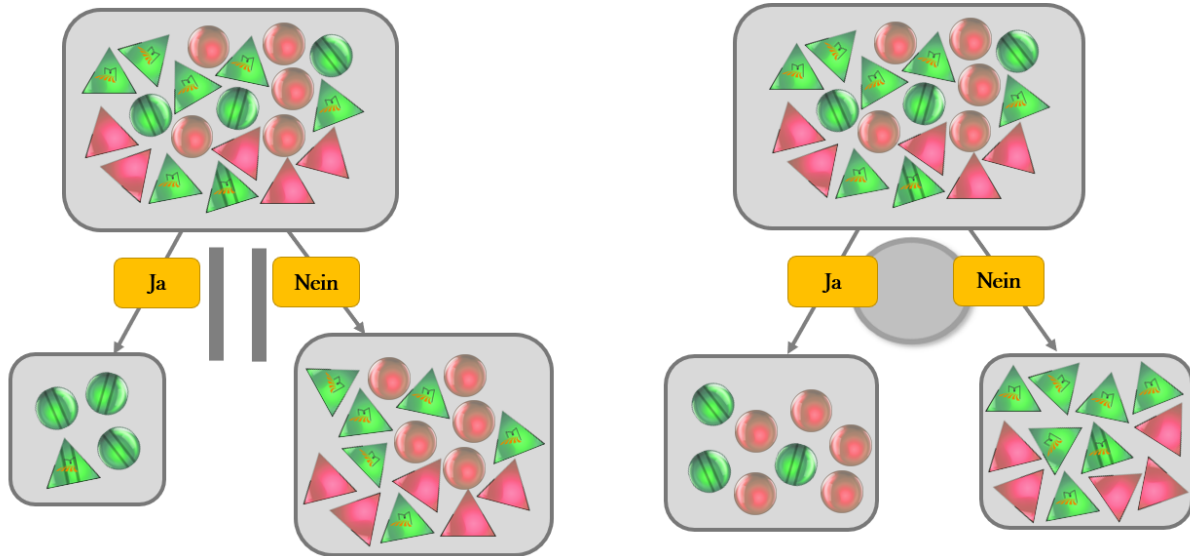
Aufteilen des Datensatzes in Trainings- und Testdaten:





1.3 Erstellen des Entscheidungsbaums anhand der Trainingsdaten

Beim Aufbau des Entscheidungsbaumes versucht man zunächst dasjenige Attribut auszuwählen, das den Datensatz in möglichst homogene Gruppen aufteilt. Folgendes Beispiel zeigt die Aufteilung eines gegebenen gelabelten Datensatzes (grün vs. rot) einmal nach dem Attribut *Streifen* und einmal nach dem Attribut *Form*:



Eine intuitive Idee, die auch in der Praxis Anwendung findet, ist es, die „Fehlerwahrscheinlichkeit“ vor und nach dem Aufteilen zu betrachten:

- In der Ausgangsmenge sind 20 Daten, die jeweils zur Hälfte grün bzw. rot sind. Wenn man sich für eines der beiden Label entscheidet, entscheidet man sich in $\frac{10}{20}$ aller Fälle falsch.
- Beim Aufteilen des Datensatzes nach dem Attribut *Streifen* entstehen zwei Teilmengen. In der ersten Teilmenge befinden sich nur grün-gelabelte Daten. Somit ist die Fehlerwahrscheinlichkeit hier $\frac{0}{4}$. In der zweiten Teilmenge befinden sich sechs grüne und zehn rote Daten. Man entscheidet sich in diesem Fall also für das rote Label und macht somit in $\frac{6}{16}$ aller Fälle einen Fehler bei der Klassifikation.
- Beim Aufteilen des Datensatzes nach dem Attribut *Form* entstehen ebenfalls zwei Teilmengen. In der ersten Teilmenge befinden sich drei grüne und fünf rote Daten. Man entscheidet sich hier also für das rote Label und erhält eine Fehlerwahrscheinlichkeit von $\frac{3}{8}$. In der zweiten Teilmenge befinden sich sieben grüne und fünf rote Daten. Man entscheidet sich in diesem Fall also für das grüne Label und begeht somit in $\frac{5}{12}$ aller Fälle einen Fehler.

Im Hinblick auf den Informationsgehalt der Teilmengen insgesamt macht es natürlich einen Unterschied, wie viele Daten jeweils in einer Teilmenge liegen. Man gewichtet die Fehlerwahrscheinlichkeit hierbei mit dem Anteil der Daten in der jeweiligen Teilmenge:

- Ausgangsmenge: $\frac{10}{20} = 0,5 = 50\%$
- Fehler nach dem Aufteilen (mit Gewichtung) nach dem Attribut *Streifen*:

$$\frac{4}{20} \cdot \frac{0}{4} + \frac{16}{20} \cdot \frac{6}{16} = \frac{0}{20} + \frac{6}{20} = \frac{6}{20} = 0,3 = 30\%$$

Informationsgewinn: „ $50\% - 30\% = 20\%$ “ (man macht nach dem Aufteilen der Datenmenge nach dem Attribut *Streifen* zu 20% weniger einen Fehler)



- Fehler nach dem Aufteilen (mit Gewichtung) nach dem Attribut *Form*:

$$\frac{8}{20} \cdot \frac{3}{8} + \frac{12}{20} \cdot \frac{5}{12} = \frac{3}{20} + \frac{5}{20} = \frac{8}{20} = 0,4 = 40\%$$

Informationsgewinn: „50 % – 40 % = 10 %“ (man macht nach dem Aufteilen der Datenmenge nach dem Attribut *Form* zu 10 % weniger einen Fehler)

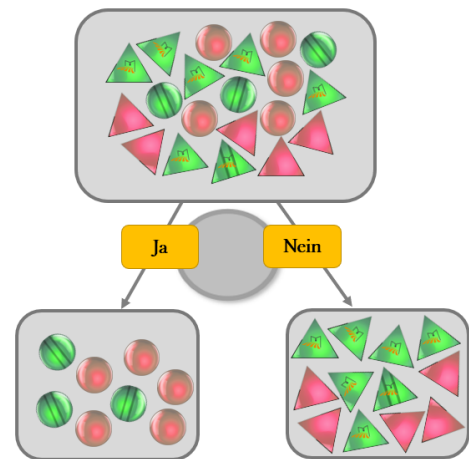
Überblick (Berechnung des Informationsgewinns anhand der Fehlklassifikationen)

Betrachtet man die obigen Berechnungen bzgl. des gewichteten Fehlers nach dem Aufteilen der Datenmenge, sieht man, dass es im Hinblick auf den Informationsgehalt ausreicht, die **Fehler in der Ausgangsmenge** bzw. die summierten **Fehler in den entstandenen Teilmengen** zu betrachten. Für die Aufteilung der Daten hinsichtlich des Attributs *Form* bedeutet dies:

- Fehleranzahl in Ausgangsmenge: 10 Fehler
- Summierte Fehleranzahl in den Teilmengen:

$$3 \text{ Fehler} + 5 \text{ Fehler} = 8 \text{ Fehler}$$

- Informationsgewinn: „10 Fehler - 8 Fehler = 2 Fehler“



Eine übersichtliche Darstellung dieses Vorgehens bietet folgende **tabellarische Notation**:

Form			
Fehler vor Aufteilen der Datenmenge:			10 Fehler
Attributwert / Label	Grün	Rot	Fehler
Rund	3	5	3 Fehler
Dreieckig	7	5	5 Fehler
Gesamt:			8 Fehler
Informationsgewinn:			10 Fehler – 8 Fehler = 2 Fehler

Bezeichnung (Das „wichtigste“ Attribut)

Das Vorgehen bei der Erstellung des Entscheidungsbaumes ist, zunächst immer das Attribut auszuwählen, das den größten Informationsgewinn liefert. Dieses Attribut wird in diesem Zusammenhang als das „wichtigste“ oder „beste“ Attribut bezeichnet.



- (c) Finden Sie nun für die beiden erhaltenen Teilmengen jeweils erneut das „wichtigste“ Attribut, indem Sie wieder die angegebenen Tabellen ausfüllen:

- linker Teilbaum:

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

„Wichtigstes“ Attribut:

- rechter Teilbaum:

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

Attribut:			
Fehler vor Aufteilen der Datenmenge:			
Attributwert / Label	<i>Friedlich</i>	<i>Feindselig</i>	Fehler
Gesamt:			
Informationsgewinn:			

„Wichtigstes“ Attribut:

- (d) Zeichnen Sie den bisher entstandenen Entscheidungsbaum:

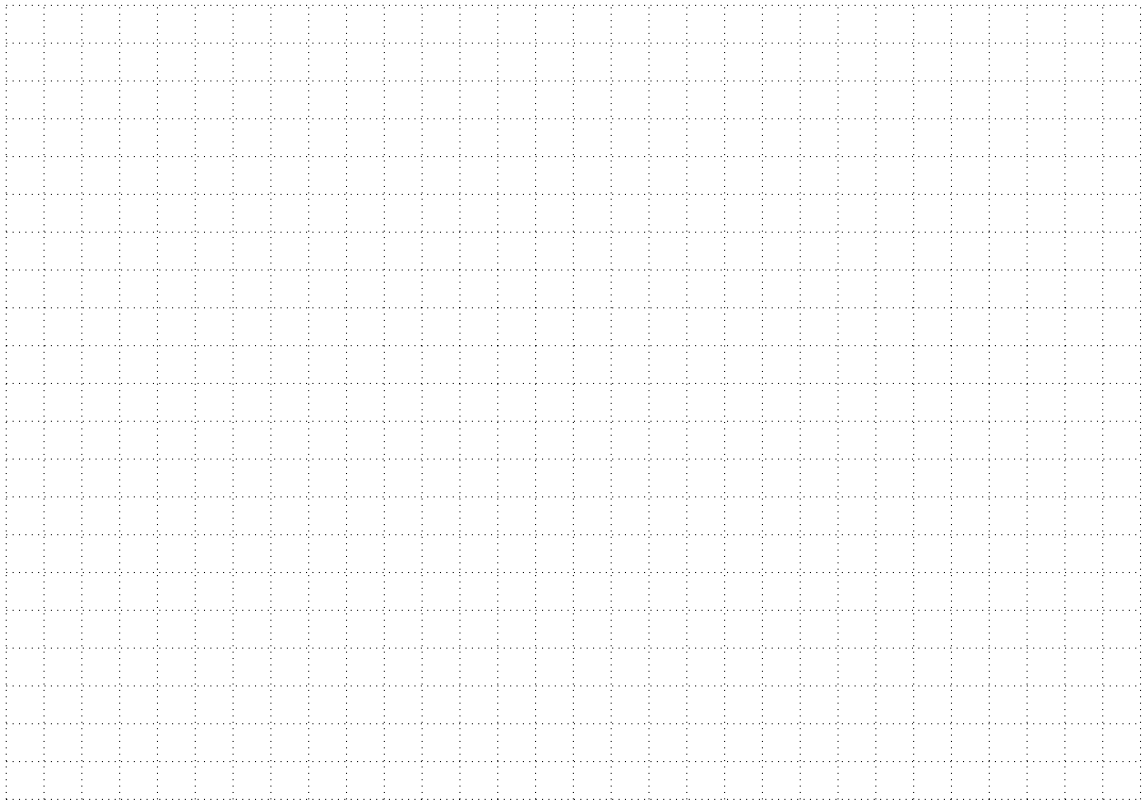


- (e) Entscheiden Sie, an welchen Stellen die entstandenen Teilmengen noch nach weiteren Attributen aufgeteilt werden müssen / sollen und finden Sie für diese Fälle wieder das „wichtigste“ Attribut. Zeichnen Sie abschließend Ihren fertigen Entscheidungsbaum:





- (f) Formulieren Sie einen Algorithmus zur Erstellung eines Entscheidungsbaumes ausgehend von einer gegebenen Menge an gelabelten Trainingsdaten:



Exkurs (Weitere Splitkriterien)

Neben der Fehlklassifikationsrate gibt es noch weitere Split-Kriterien, die in der Praxis bevorzugt eingesetzt werden, da diese im Vergleich zur Fehlklassifikationsrate gewisse Vorteile bieten. Aus unserer Sicht ist im Hinblick auf die Zielgruppe die Fehlklassifikationsrate den anderen beiden Split-Kriterien zur Bestimmung des Informationsgewinns vorzuziehen.

(a) Entropie

Die Entropie einer Menge X mit k verschiedenen Merkmalsklassen (mit jeweiliger Auftretenswahrscheinlichkeit p_i , $i = 1, \dots, n$) wird wie folgt berechnet:

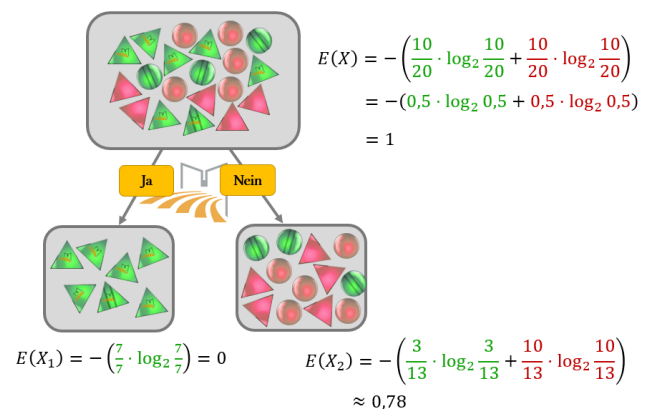
$$E(X) = - \sum_{i=1}^k p_i \cdot \log_2(p_i)$$

Bei zwei Merkmalsausprägungen (z.B. grün/rot oder Punkte / keine Punkte):

$$E(X) = -(p_1 \cdot \log_2(p_1) + p_2 \cdot \log_2(p_2))$$

Der Informationsgewinn ist demnach für das Aufteilen der Datenmenge nach dem Attribut *Logo*:

$$\text{IG}_E(X, \text{Logo}) = E(X) - \frac{7}{20} \cdot E(X_1) - \frac{13}{20} \cdot E(X_2) \approx 1 - \frac{13}{20} \cdot 0,78 \approx 0,493$$



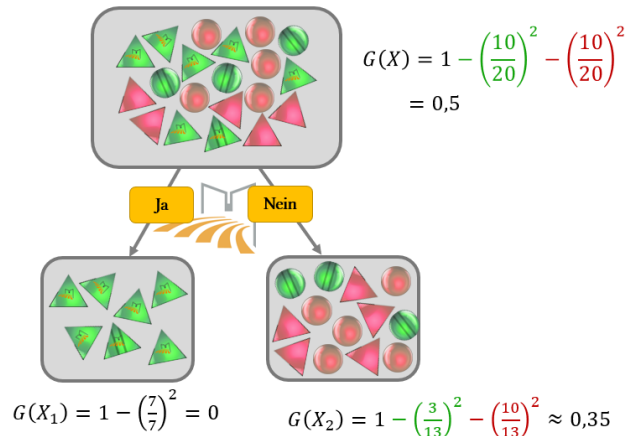
(b) **Gini-Impurity:**

Die **Gini-Impurity** einer Menge X mit k verschiedenen Merkmalsklassen (mit jeweiliger Auftretenswahrscheinlichkeit p_j , $j = 1, \dots, n$) wird wie folgt berechnet:

$$G(X) = 1 - \sum_{i=1}^k p_i^2$$

Bei zwei Merkmalsausprägungen:

$$G(X) = 1 - p_1^2 - p_2^2$$



Der Informationsgewinn ist demnach für das Aufteilen der Datenmenge nach dem Attribut *Logo*:

$$IG_G(X, \text{Logo}) = G(X) - \frac{7}{20} \cdot G(X_1) - \frac{13}{20} \cdot G(X_2) \approx 1 - \frac{13}{20} \cdot 0,35 \approx 0,77$$

1.4 Testen des Entscheidungsbaumes

Nachdem wir im vorangegangenen Abschnitt ein Verfahren kennengelernt haben, mit dem man ausgehend von einer Menge an gelabelten Trainingsdaten einen Entscheidungsbaum erstellen kann, geht es im Anschluss darum, die „Vorhersage-Qualität“ des generierten Entscheidungsbaums mithilfe der Testdaten einzuschätzen.

Arbeitsauftrag 3: Klassifikation der Testdaten

Von ursprünglich allen Fischen mit bekannten Label haben wir folgende fünf Fische als Testdaten deklariert, d.h. diese wurden nicht zur Erstellung des Entscheidungsbaumes verwendet:



- (a) Klassifizieren Sie die fünf Fische gemäß Ihres erstellten Entscheidungsbaumes und vergleichen Sie das vorhergesagte Label mit dem tatsächlich Label. Eine gute Übersicht über die richtigen und falschen Klassifikationen bietet folgende Tabelle:

Erwartetes Label		Berechnetes Label		Summe
		Friedlich	Feindselig	
	Friedlich			
	Feindselig			
	Summe			



(b) Beurteilen Sie die „Vorhersage-Qualität“ des Entscheidungsbaumes anhand obiger Tabelle:

Überblick (Die Konfusionsmatrix)

Eine gute Übersicht über die Klassifikation der Testdaten mithilfe des erstellten Entscheidungsbaumes bietet die sog. **Konfusionsmatrix**. Im Falle einer binären Klassifikation zeigt diese die absoluten Zahlen der richtig positiv, richtig negativ, falsch positiv bzw. falsch negativ klassifizierten Testdaten:

Erwartetes Label	Berechnetes Label			
	Friedlich	Feindselig		
	Friedlich	3	0	
	Feindselig	1	1	2
	4	1		

Die **Genauigkeit** der Vorhersage des Entscheidungsbaumes wird als Anteil der richtig klassifizierten Testdaten in Relation zur Gesamtzahl der Testdaten angegeben. Im vorliegenden Beispiel bedeutet dies:

$$\frac{1 + 3}{1 + 1 + 3 + 0} = \frac{4}{5} = 80\%$$

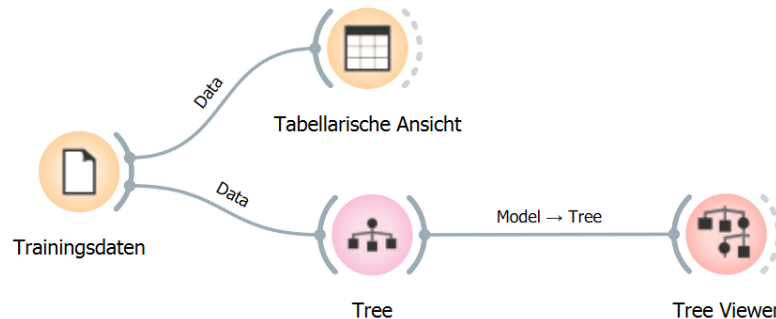
1.5 Entscheidungsbäume in Orange²

Da das Erstellen von Entscheidungsbäumen von Hand insbesondere bei sehr großen Datenmengen äußerst mühsam ist, bedarf es hier einer geeigneten Software. Für den Schulunterricht bietet sich die professionelle Data-Mining und Machine-Learning Software **Orange** an. Hierbei handelt es sich um ein sehr umfangreiches Tool, welches viele Funktionen in Form von Widgets bereitstellt. Unter gewissen Voraussetzungen (insbesondere einer kleinschrittigen Anleitung der benötigten Widgets) erachten wir diese Software für den Unterrichtseinsatz geeignet, um insbesondere den Einfluss von Trainingsdaten und Hyperparametern auf die „Vorhersage-Qualität“ des Entscheidungsbaumes veranschaulichen zu können.

²Ein ähnliches Vorgehen findet sich bei „Von Daten und Bäumen“ (<https://computingeducation.de/proj-it2school/>).

**Arbeitsauftrag 4: Entscheidungsbaum zum kleinen Fischdatensatz**

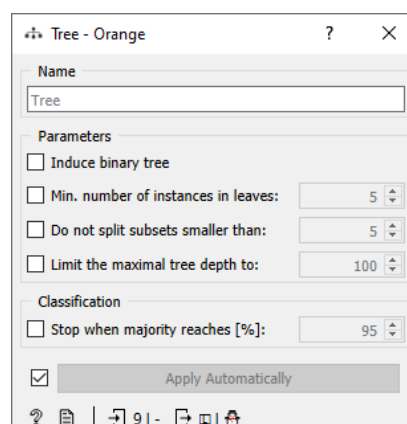
Ziehen Sie via Drag & Drop die folgenden Widgets auf die Arbeitsfläche und „verkabeln“ Sie diese entsprechend der Abbildung:



- (a) Laden Sie aus dem Mebis-Kurs den Ordner **Material Entscheidungsbäume** herunter. 
- (b) Unter **File** können Sie eine **csv**-Datei einlesen. Wählen Sie hier zunächst die Datei **Datensatz_Fisch_Einstieg_Trainingsdaten.csv** aus. Wichtig ist, dass Sie dann noch die Spalte kennzeichnen, die das Label der Daten darstellen soll. Diese Spalte (in unserem Beispiel die Spalte mit den Einträgen *friedlich* / *feindselig*) muss als **target** markiert sein:

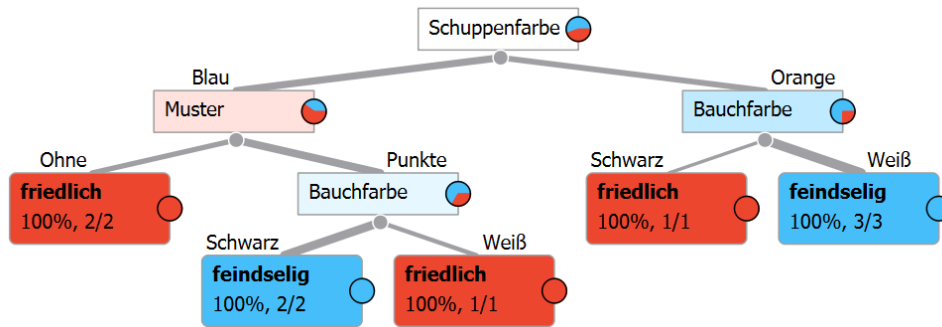
Columns (Double click to edit)				
	Name	Type	Role	Values
1	Muster	 categorical	feature	Ohne, Punkte
2	Bauchfarbe	 categorical	feature	Schwarz, Weiß
3	Schuppenfarbe	 categorical	feature	Blau, Orange
4	Flossenfarbe	 categorical	feature	Gelb, Rot
5	Label	 categorical	target	feindselig, friedlich

- (c) Das Widget **Data Table** (Tabellarische Ansicht) ist an sich unnötig, es bietet aber die Möglichkeit, den importierten Datensatz direkt in Orange betrachten zu können.
- (d) Das Widget **Tree** erstellt anhand verschiedener Hyperparameter den Entscheidungsbaum auf Grundlage der übergebenen Daten. Da wir in unserem Einstiegsbeispiel nur sehr wenig Daten haben, wählen wir folgende Konfiguration:



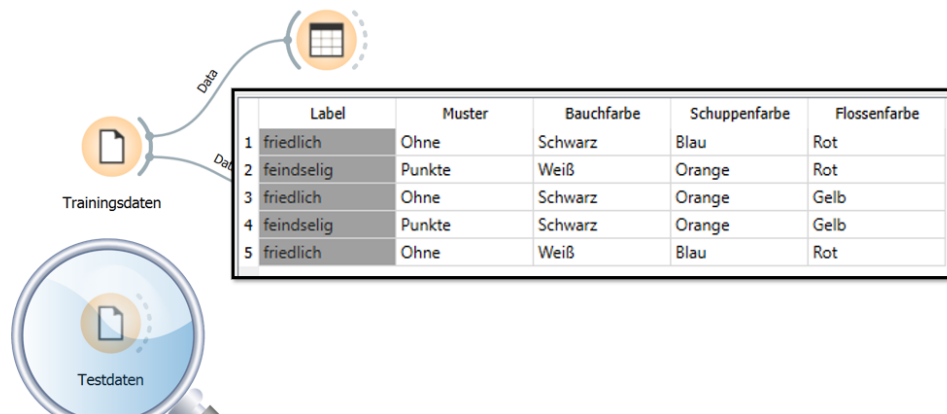


- (e) Mit dem Widget **Tree Viewer** können wir uns den erstellten Entscheidungsbaum graphisch anzeigen lassen. Wir deaktivieren hierzu noch die Checkbox **Show details in non-leaves**:

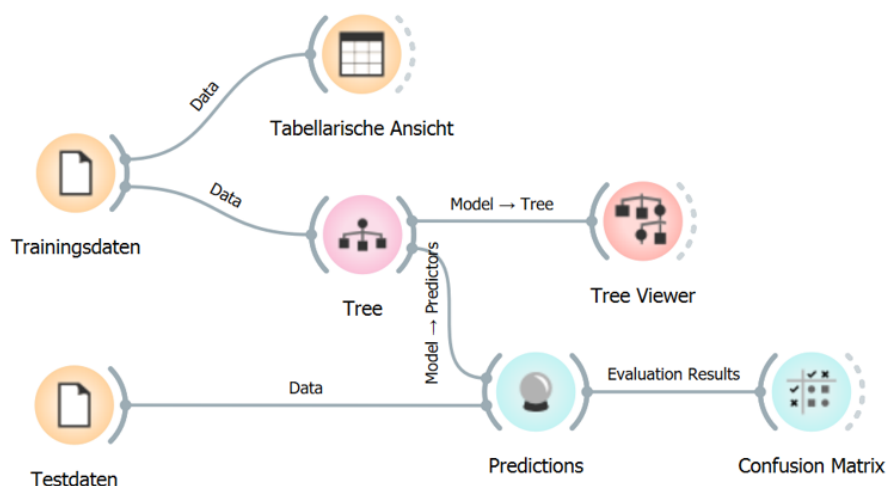


Wir erhalten via Orange also denselben Entscheidungsbaum, den wir vorab manuell erstellt haben. Da Orange mit der Entropie ein anderes Splitkriterium verwendet als wir mit der Fehlklassifikationsrate, ist dies im vorliegenden Fall tatsächlich Zufall.

- (f) Auch das Testen anhand der Testdaten kann in Orange umgesetzt werden. Wir importieren dazu zunächst die Testdaten `Datensatz_Fische_Einstieg_Testdaten.csv` in einem eigenen Widget:

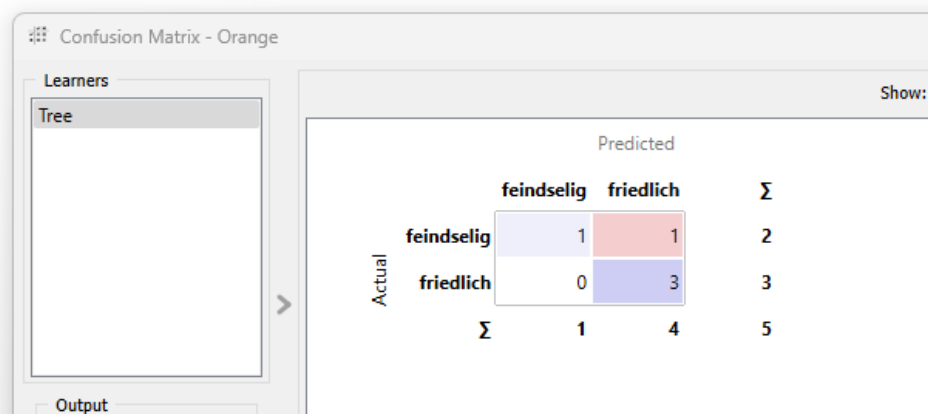


- (g) Mithilfe des Widgets **Predictions** können wir das erstellte Entscheidungsbaum-Modell anhand der Testdaten überprüfen. Das Ergebnis lassen wir uns in der Konfusionsmatrix anzeigen:

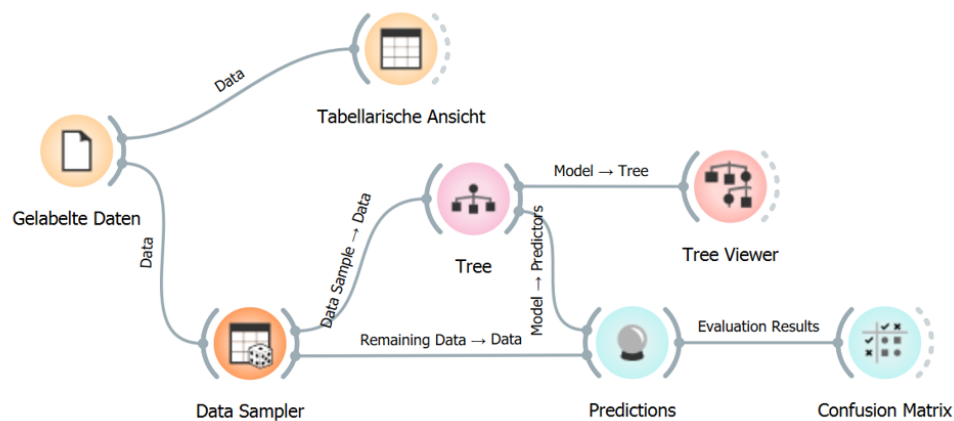




- (h) Die Inhalte der von Orange erstellten Konfusionsmatrix decken sich erwartungsgemäß mit der von uns manuell erstellten Auswertung:



- (i) Wir können Orange alle vorhandenen gelabelten Daten auch zufällig in Trainings- und Testdaten aufteilen lassen. Löschen Sie hierzu das Widget mit den Testdaten und importieren Sie unter dem Widget Trainingsdaten die Datei `Datensatz_Fische_Kleiner_Datensatz.csv`. Diese beinhaltet sowohl die bisherigen Trainings- als auch Testdaten. Als nächstes benötigen wir das Widget `DataSampler`. Mit diesem Widget können wir festlegen, welcher Prozentsatz oder welche absolute Zahl der übergebenen Datenmenge genutzt werden soll. Wir können somit einen bestimmten Wert der Daten als Trainingsdaten verwenden. Die restlichen Daten verwenden wir als Testdaten. Hierzu müssen wir noch auf der Verbindung zwischen dem `DataSampler` und dem Widget `Predictions` die Linie zwischen `Remaining Data` und `Data` ziehen:



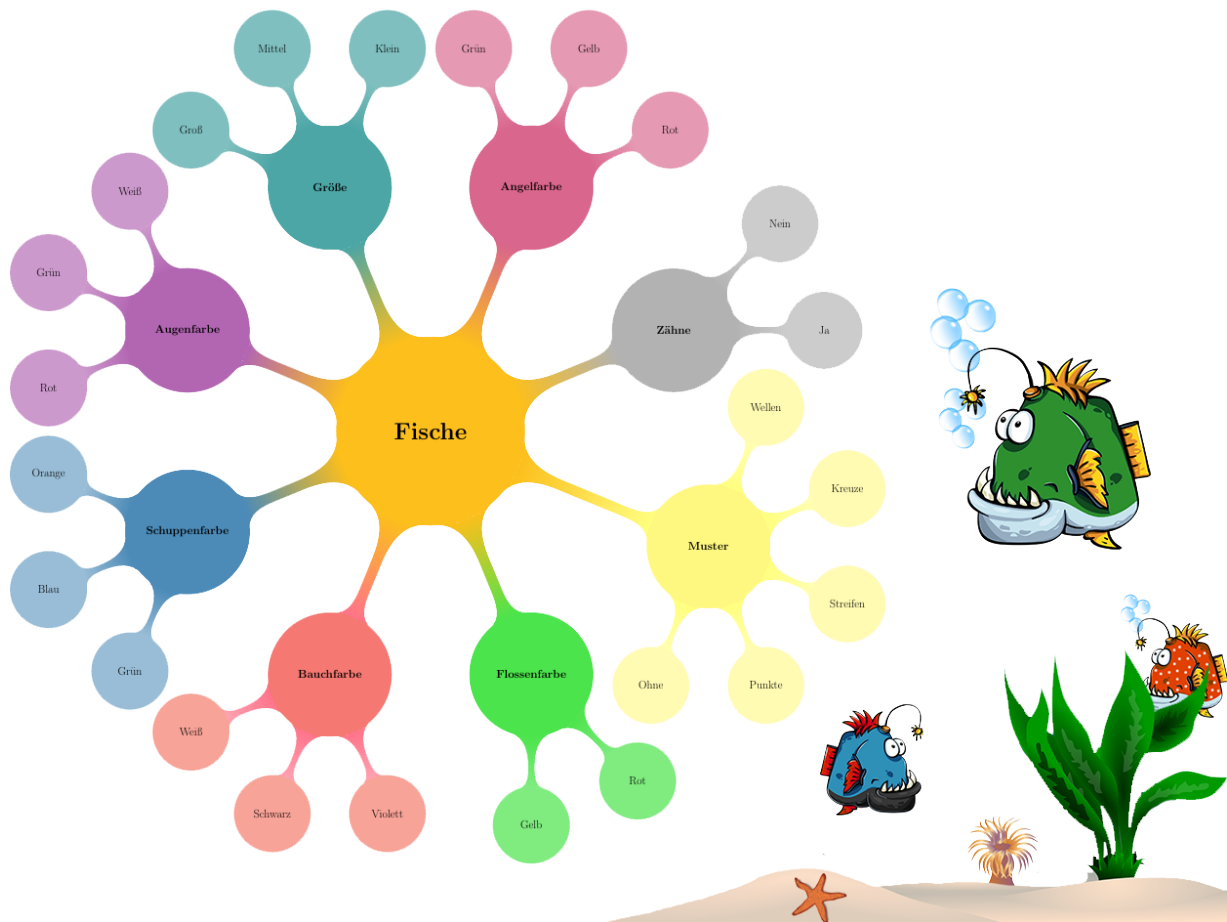
- (j) Experimentieren Sie mit verschiedenen Aufteilungen zwischen Trainings- und Testdaten sowie den im Widget `Tree` möglichen Einstellungen und betrachten Sie die Auswirkungen auf den Entscheidungsbaum bzw. die Konfusionsmatrix.





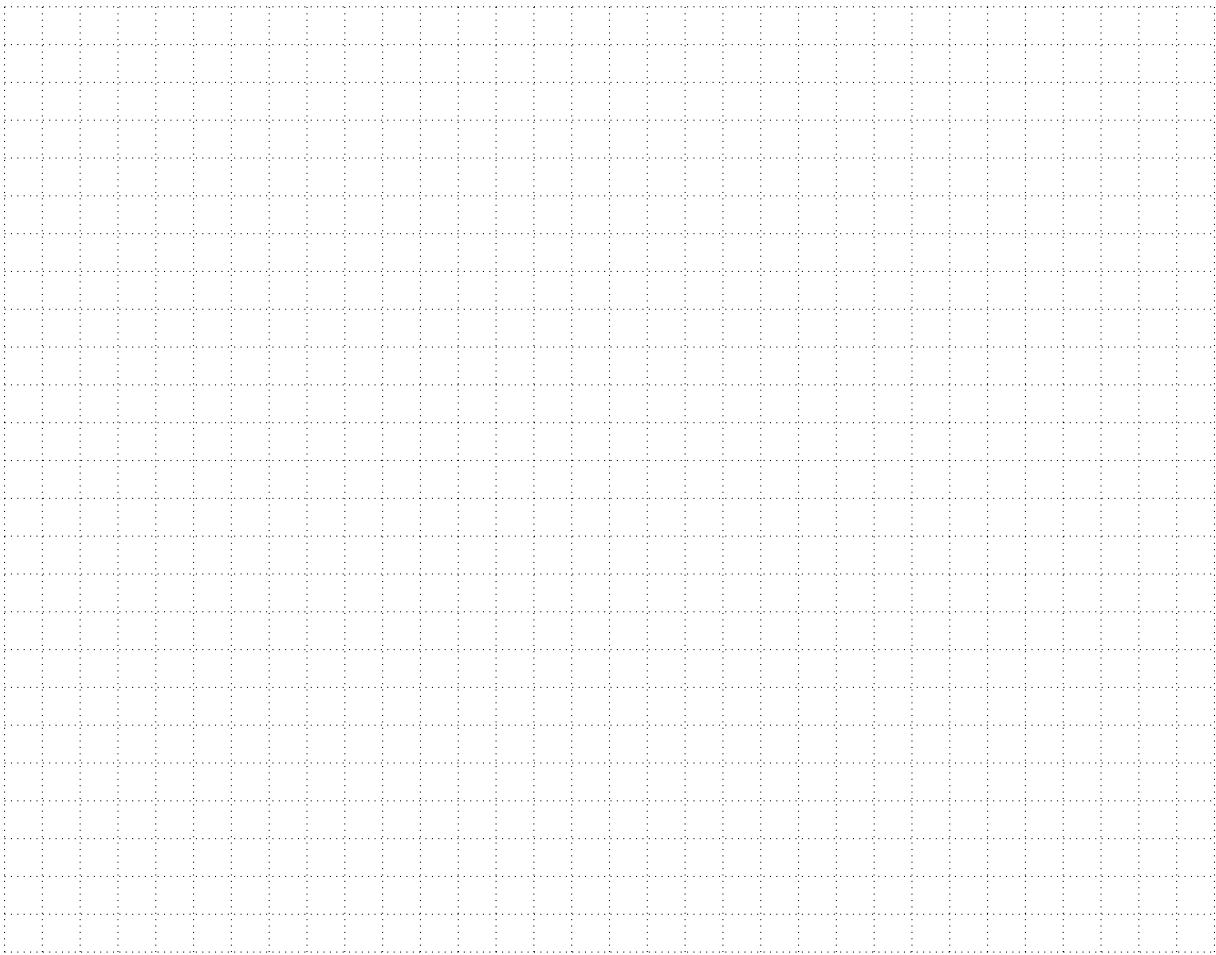
Arbeitsauftrag 5: Entscheidungsbaum zum großen Fischdatensatz

Die CSV-Datei `Datensatz_Fische_Großer_Datensatz.csv` beinhaltet einen großen Fischdatensatz mit 1626 Fischen und mehr Merkmalen bzw. Merkmalsausprägungen als bisher. Eine Übersicht über die Merkmale bietet folgende Mindmap:



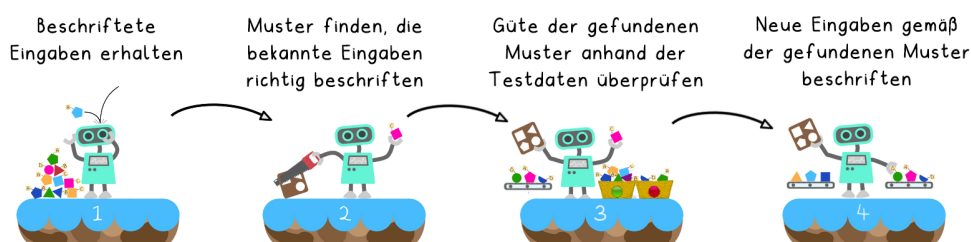
Erstellen Sie mithilfe der Software Orange einen Entscheidungsbaum zum Fischdatensatz. Betrachten Sie die Auswirkungen der Wahl verschiedener Hyperparameter (Aufteilung Trainings- und Testdaten, Baumtiefe, Splitabbruchbedingung etc.) auf die „Vorhersage-Qualität“ des Entscheidungsbaumes anhand der Konfusionsmatrix. Zeichnen Sie den Ihrer Meinung nach „besten Baum“ in das Arbeitsheft:





Überblick (Entscheidungsbäume)

- der Entscheidungsbaum-Algorithmus sucht gemäß dem Verfahren des überwachten Lernens nach Zusammenhängen in gelabelten Daten und erstellt daraus ein Modell.
- **Aber:** hierbei handelt es sich lediglich um ein heuristisches Vorgehen und es gibt keine Garantie, dass der „beste“ bzw. überhaupt „ein guter“ Entscheidungsbaum gefunden wird.
- der Algorithmus ist nur dann sinnvoll anwendbar, wenn in den Daten Korrelationen vorhanden sind.
- Hyperparameter wie Baumtiefe oder prozentuale Aufteilung in Trainings- und Testdaten nehmen starken Einfluss auf den Entscheidungsbaum.
- die „Vorhersage-Qualität“ des Entscheidungsbaumes kann mit der Konfusionsmatrix eingeschätzt werden.



Adaptiert von „Überwachtes Lernen“ (<https://computingeducation.de/proj-ml-uebersicht/>) (CC-BY) von Stefan Seegerer, Tilman Michaeli & Sven Jatzlau.

Wolfgang Pfeffer
Tobias Fuchs
Ute Heuer



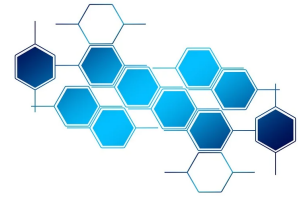


2. Der k -nächste-Nachbarn-Algorithmus

2.1 Kompetenzen laut Lehrplan

Kompetenzerwartungen Informatik 11 (NTG / *spät beginnend*):

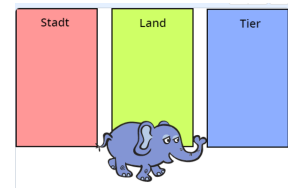
- erläutern die **Funktionsweise** / **Idee** eines ausgewählten Algorithmus maschinellen Lernens (k -nächste-Nachbarn- oder Entscheidungsbaum-Algorithmus) **allgemein und** an konkreten Beispielen.
- analysieren den Einfluss von Trainingsdaten und Parametern auf die Zuverlässigkeit der Ergebnisse eines Verfahrens maschinellen Lernens, ggf. unter Verwendung eines geeigneten Werkzeugs.



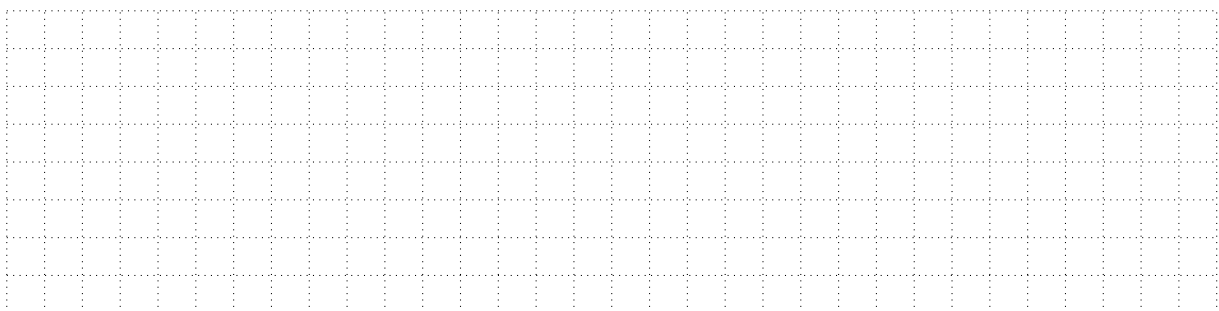
2.2 "Was bin ich? Stadt, Land, Tier-Edition" - KNN-Algorithmus und Scratch

Arbeitsauftrag 6: KNN-Algorithmus in der Scratch-Umgebung

Wir nutzen als Einstieg ein kleines Quiz-Programm, welches eine Eingabe des Nutzers mithilfe des KNN-Algorithmus den Klassen *Stadt*, *Land* oder *Tier* zuordnet. Mit Hilfe eines verhältnismäßig kleinen Trainingsdatensatzes von nur 60 Daten (je 20 pro Klasse) lassen sich bereits sehr beeindruckende Ergebnisse in der Vorhersage der korrekten Klasse erzielen.



- Laden Sie aus dem Mebis-Kurs den Ordner  **Material_KNN** herunter.
- Rufen Sie die Scratch-Umgebung unter <https://playground.raise.mit.edu/intent-classifier/> auf.
- Importieren Sie die Projektdatei aus dem Ordner **KNN_Scratch**, sodass nebenstehendes Programm angezeigt wird.
- Öffnen Sie die Trainingsdaten aus dem Ordner **KNN_Scratch** in der Umgebung.
- Probieren Sie das Programm aus, indem Sie unterschiedliche Städte, Länder und Tiere eingeben.





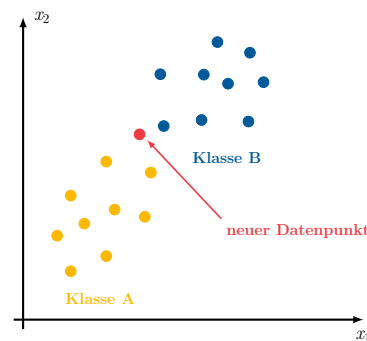
2.3 Klassifikation mit dem k -nächste-Nachbarn-Algorithmus

Überblick (Klassifikation mit Hilfe des KNN-Algorithmus)

Der **k -nächste-Nachbarn-Algorithmus (KNN)** gehört zum Teilgebiet des überwachten maschinellen Lernens und kann u.a. zur Klassifikation verwendet werden. KNN versucht dabei die richtige Klasse für einen neuen Datenpunkt vorherzusagen, indem der **Abstand** zwischen dem neuen Datenpunkt und den gelabelten Trainingspunkten berechnet wird. Im Anschluss werden die bezüglich des berechneten Abstands k nächsten Punkte ausgewählt. Der zu klassifizierende Datenpunkt wird der Klasse zugeordnet, von der sich am meisten Datenpunkte unter den k nächsten Punkten befinden.

Ausgangssituation:

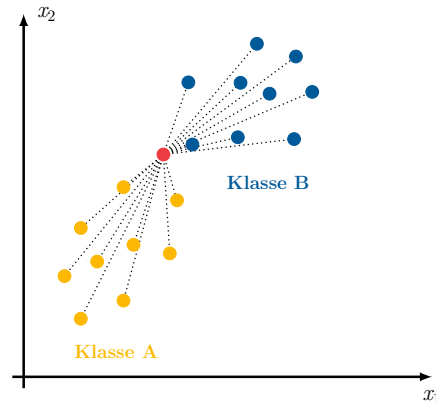
Unsere Trainingsdaten sind in zwei Klassen unterteilt, die im Folgenden als orange und blaue Punkte ($x_1 \mid x_2$) visualisiert werden. Ausgehend von diesem Trainingsdatensatz soll der KNN-Algorithmus einen neuen Datenpunkt (rot) klassifizieren, von dem die Klasse nicht bekannt ist.



Vorgehen des KNN-Algorithmus:

- (a) *Bestimmung der Abstände des neuen Datenpunktes zu allen Trainingsdaten und Auswahl der k Datensätze mit dem kleinstem Abstand*

Für unsere Trainingsdaten wird nun jeweils der Abstand zum neuen Datenpunkt bestimmt. Für die Abstandsberechnung können unterschiedliche **Abstandsmaße**, wie etwa der euklidische Abstand (siehe unten), verwendet werden. Nachdem die Abstände zu allen Trainingsdatenpunkten berechnet wurden, werden die bezüglich des berechneten Abstands nächsten k Trainingsdatenpunkte als k nächste Nachbarn ausgewählt.



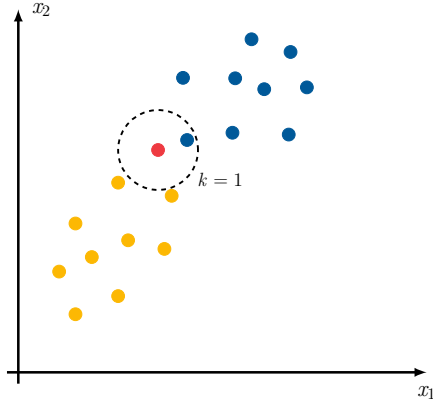
Hinweis:

Wir bestimmen an dieser Stelle die Abstände des zu klassifizierenden Datenpunktes zu allen Trainingsdaten. Für weiterführende Informationen zu optimierten Abstandsberechnungen und der Auswahl der dafür herangezogenen Trainingsdaten empfehlen wir als Exkurs das **Buzzword** Locality-Sensitive Hashing.

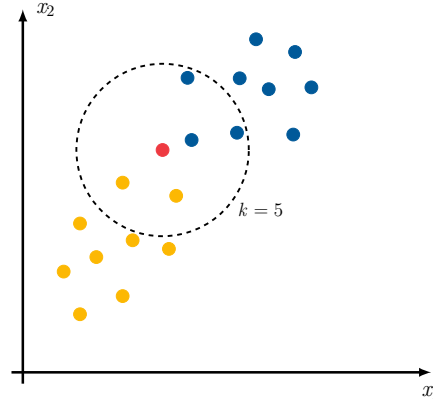


- (b) *Einordnung des neuen Datenpunkts in die mehrheitlich vorkommende Klasse der bestimmten k nächsten Nachbarn*

Zur Klassifikation des neuen Datenpunkts werden nun die Klassen der gefundenen k nächsten Nachbarn betrachtet. Die mehrheitlich unter diesen Nachbarn vorkommende Klasse wird als Klasse des neuen Datenpunkts gewählt.

Parameter $k = 1$:

Wegen $k = 1$ wird nur der dem neuen Datenpunkt am nächsten liegende Trainingsdatenpunkt betrachtet. Da dieser zur Klasse blau gehört, wird der neue Datenpunkt blau klassifiziert.

Parameter $k = 5$:

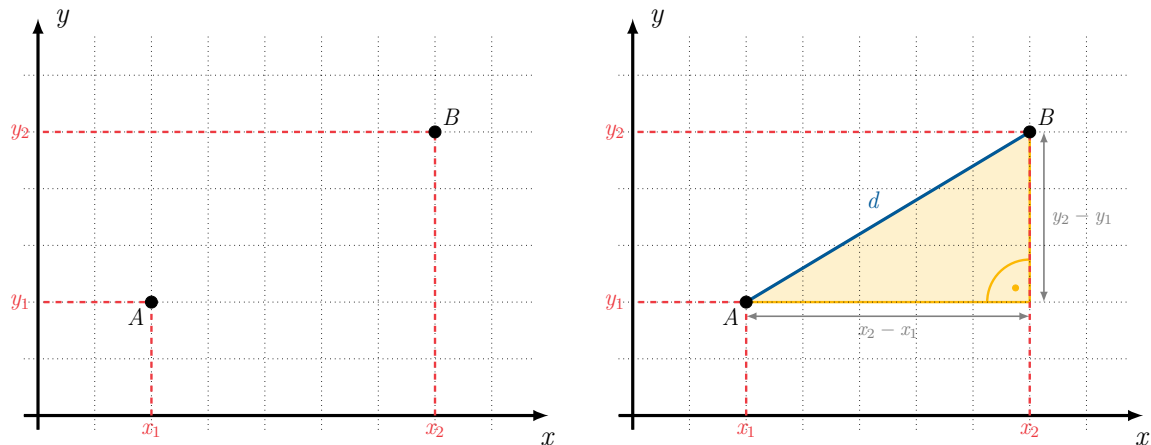
Für $k = 5$ werden die fünf nächstgelegenen Trainingsdatenpunkte betrachtet. Von diesen gehören drei zur Klasse blau und zwei zur Klasse orange. Der neue Datenpunkt wird also als blau klassifiziert.

Überblick (Gängige Abstandsmaße)

Nachdem wir mit der grundlegenden Funktionsweise des KNN-Algorithmus vertraut sind, beschäftigen wir uns nun genauer damit, wie der Abstand zwischen zwei Datenpunkten berechnet werden kann.

(a) Der Euklidische Abstand

Die für uns intuitivste Methode den Abstand zwischen zwei Punkten zu berechnen, ist der **Euklidische Abstand**. Liegen die Datenpunkte in der Form $(x \mid y) \in \mathbb{R}^2$ vor, lassen sich diese gut in einem zweidimensionalen Koordinatensystem visualisieren. Der Abstand der beiden Punkte A und B entspricht dann der Länge d der Strecke $[AB]$ (blau):



Die Länge der Strecke d lässt sich mit dem Satz des Pythagoras berechnen:

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2 \iff d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Diese Formel lässt sich auch in den $\mathbb{R}^3$ bzw. allgemein auf den $\mathbb{R}^n$ (Datenvektor mit n Einträgen) übertragen.

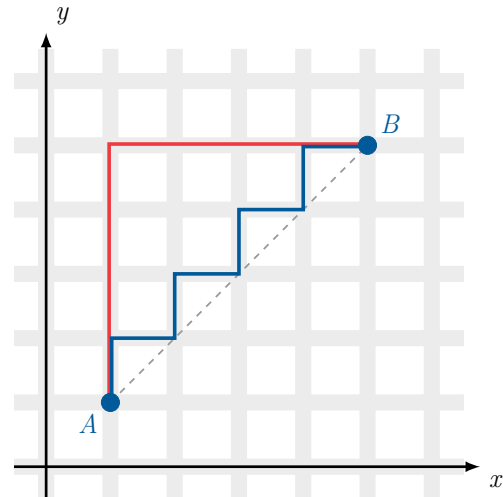


(b) Die Manhattan-Metrik

Die Manhattan-Metrik hat ihren Namen von dem schachbrettartigen Straßenmuster in Manhattan. Um dort von Punkt A nach Punkt B zu kommen, muss man den Straßen folgen und kann nicht die direkte Verbindung (euklidischer Abstand) wählen.

Gegeben sind die Punkte $A = (x_1 | y_1)$ und $B = (x_2 | y_2)$. Dann ist der Abstand d (rot bzw. blau) definiert als die Summe der absoluten Differenzen der Einzelkoordinaten von A und B:

$$d = |x_2 - x_1| + |y_2 - y_1|$$

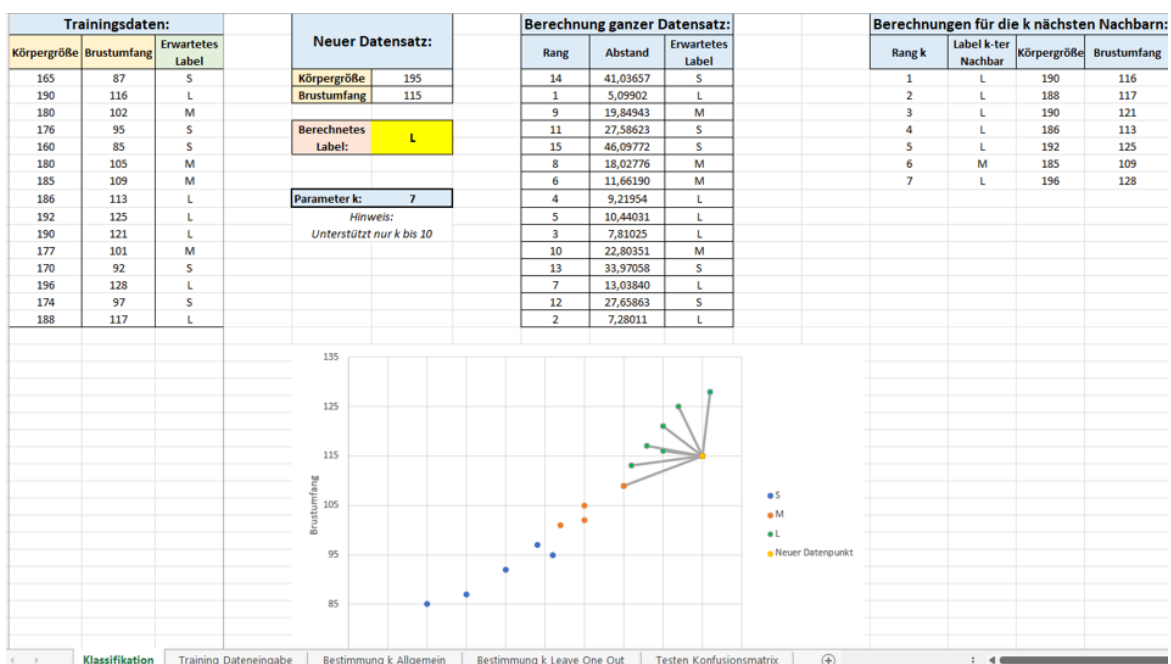


Anwendung (Klassifikation von Schulshirtgrößen)

Als mögliche Anwendung des KNN-Algorithmus wurde eine Excel-Mappe zur Vorhersage der passenden Shirtgröße für folgenden Schulshirt-Kontext entwickelt: Am KI Gymnasium (KIG) wurden von Herrn Turing Schulshirts für seine 11. Jahrgangsstufe entworfen. Damit nicht so viele Größen (S, M, L, XL, XXL, XXXL) produziert werden müssen, hat Herr Turing die Größen neu eingeteilt und es gibt nur die Größen S, M und L, welche alle oberen Größen abdecken.



Herr Turing hat aber leider das KIG verlassen und dabei vergessen seine Einteilung in die neuen Größen S, M und L zu hinterlassen. Lediglich die Daten seiner 11 Jahrgangsstufe sind noch vorhanden. Es wurde dabei die **Körpergröße**, der **Brustumfang** und die daraufhin gekaufte **Shirt-Größe** notiert.





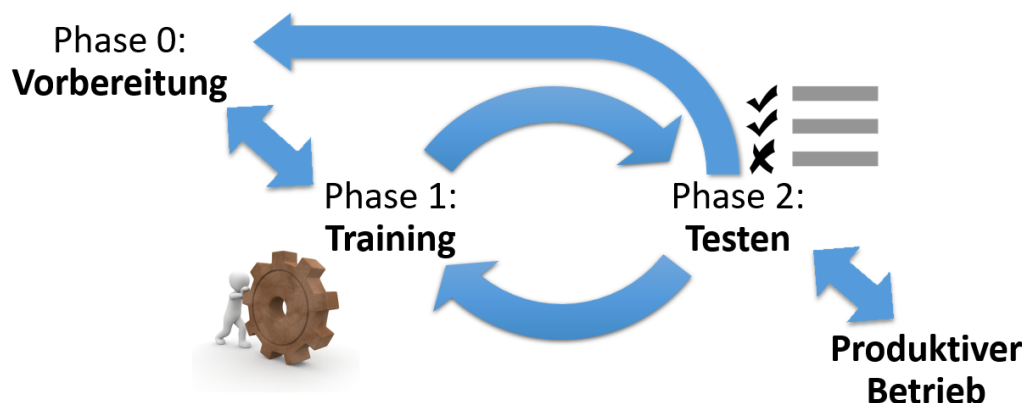
Mit Hilfe der vorliegenden Daten soll der KNN-Algorithmus genutzt werden, um auf Basis der bekannten Merkmale Körpergröße und Brustumfang die passende Shirtgröße zu prognostizieren. Die Mappe ist so aufgebaut, dass sie direkt zur Klassifikation verwendet werden kann, besitzt aber auch weitere Tabellen um wichtige Aspekte der Phasen des maschinellen Lernprozesses (u.a. Bestimmung eines passenden Werts für k oder das Aufstellen der Konfusionsmatrix in der Testphase) genauer betrachten und ausprobieren zu können.

Arbeitsauftrag 7: Ergänzen der Abstandsberechnung

- Öffnen Sie den Ordner `KNN_TKS`.
- Öffnen Sie die Mappe `k_naechster_Nachbar_Klassifikation_Datensatz_T_Shirt` und navigieren Sie zur Tabelle `Klassifikation`.
- Die Spalte `Abstand` ist noch nicht gefüllt. Ergänzen Sie nacheinander passende Formeln zur Berechnung des Abstands mit der/dem
 - Manhattan-Metrik
 - Euklidischen Abstand

2.4 Der Lernprozess des KNN-Algorithmus

Ein KI-System, welches maschinelles Lernen nutzt, durchläuft einige Phasen bevor es in den produktiven Betrieb gehen kann. Diese Phasen sind bei den meisten Systemen sehr ähnlich:





2.4.1 Phase 0: Vorbereitung

Vor Beginn des Lernprozesses sind oft einige Vorbereitungen zu treffen bevor mit dem Training begonnen werden kann. Im Folgenden sind einige Schritte der Vorbereitungsphase aufgelistet:

- **Festlegen von Hyperparameter(n)³:**

Zu Beginn müssen die Hyperparameter festgelegt werden. Beim KNN-Algorithmus muss hierbei der Parameter k festgelegt werden, bevor mit dem Training begonnen werden kann. Dies kann allerdings auch automatisiert erfolgen (vgl. Abschnitt 2.4.3).

- **Vorverarbeitung der Eingabedaten:**

Um neue Datenpunkte klassifizieren zu können, müssen die Eingabedaten mit zugehörigen Labels belegt werden. Falls die Eingabedaten bereits alle über ein Label verfügen, so wie es bei unseren Beispielen immer der Fall war, entfällt dieser Schritt.

- **Aufteilung der Daten:**

Im Verlauf des maschinellen Lernprozesses werden verschiedene Phasen durchlaufen. Für diese Phasen werden jeweils Daten benötigt. Deshalb müssen die zur Verfügung stehenden gelabelten Daten für die verschiedenen Zwecke aufgeteilt werden.

- *Trainingsdaten:*

Diese Daten werden zum Training des Modells verwendet, wie etwa zum Erstellen eines Entscheidungsbaums oder zum Anpassen der Gewichte beim Perzeptron

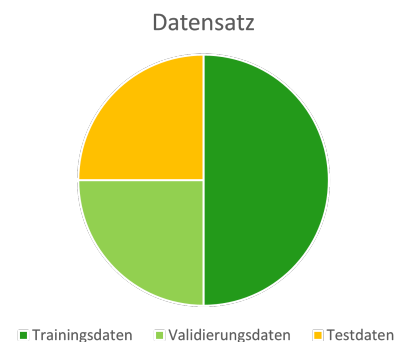
- *Validierungsdaten:*

Diese Daten werden zur Abstimmung bzw. Optimierung der Hyperparameter verwendet

- *Testdaten:*

Diese Daten werden zur Prüfung der Güte des trainierten Modells verwendet

- ...



2.4.2 Phase 1: Training

Beim k -nächste-Nachbarn-Algorithmus ist die Trainingsphase sehr einfach. Sie besteht lediglich aus dem Laden der Trainingsdaten. Damit ist das Modell trainiert.

Arbeitsauftrag 8: Trainieren des KNN-Modells

- Öffnen Sie die Datei `Trainingsdaten_KNN`
- Öffnen Sie die Excel-Mappe `k_naechster_Nachbar_Klassifikation_Datensatz_T_Shirt` und navigieren Sie zur Tabelle `Training_Dateneingabe`
- Kopieren Sie die Trainingsdaten aus (a) in die Excel-Tabelle aus (b). Prüfen Sie, ob die Daten automatisch in die anderen Tabellen der Excel-Mappe aus (b) übernommen werden.

³Dieser Schritt kann auch Phase 1 zugeordnet werden.



2.4.3 Bestimmung eines passenden Werts für k

Der Parameter k kann als Hyperparameter angesehen werden. Hierbei stellt sich allerdings die Frage, welchen Wert man für k wählen sollte, um ein möglichst gutes Modell zu erhalten. Mit geeigneten Verfahren kann die Wahl des Parameters k optimiert werden.

Überblick (Bestimmung von k anhand der Validierungsdaten)

Der Hyperparameter k legt fest, wie viele Nachbarn des zu klassifizierenden Datenpunkts aus den Trainingsdaten verwendet werden, um dessen Klasse zu bestimmen. Da dies einen großen Einfluss auf die Auswahl der Klasse des zu klassifizierenden Datenpunkts haben kann, sollte der Wert für k passend gewählt werden. Zur Optimierung der Wahl von k können die Validierungsdaten verwendet werden.

Vorgehen:

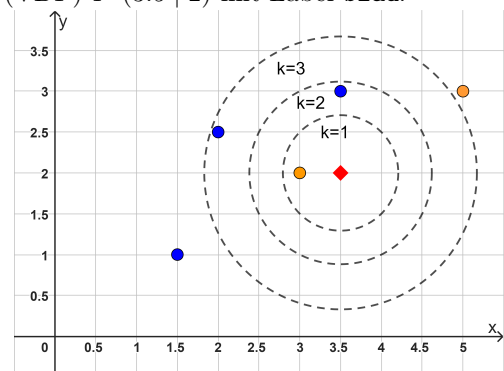
- Auswahl eines Validierungsdatenpunkts P
- Berechnung der Klassifikation für P mit Hilfe des KNN-Modells für verschiedene Werte von k
- Notieren für welche k das berechnete Label mit dem tatsächlichen Label von P übereinstimmt
- Wiederholen des Vorgehens für alle Datenpunkte des Validierungsdatensatzes
- Auswahl des Wertes für k , der am häufigsten zu einer korrekten Vorhersage des erwarteten Labels geführt hat

Beispiel:

Gegeben sind die Trainingsdaten der Klasse **orange** (3 | 2) und (5 | 3), sowie (3.5 | 3), (2 | 2.5) und (1.5 | 1) der Klasse **blau**. Wir betrachten den Validierungsdatenpunkt (VDP) P (3.5 | 2) mit Label **blau**.

P wird nun für verschiedene Werte von k (hier $k = 1, 2, 3$) mit dem KNN-Modell klassifiziert und das berechnete Label wird mit dem erwarteten Label von P verglichen. Die Ergebnisse sind in folgender Tabelle dargestellt:

	$k = 1$	$k = 2$	$k = 3$	...
VDP P	falsch	falsch	korrekt	...
...	...	...	...	...



Für $k = 3$ werden die meisten Validierungsdaten (hier bisher nur P) korrekt vorhergesagt. Wir wählen also $k = 3$ für unseren Parameter k .

Arbeitsauftrag 9: Optimierung der Wahl von k

- Öffnen Sie die Excel-Mappe `k_naechster_Nachbar_Klassifikation_Datensatz_T-Shirt` und navigieren Sie zur Tabelle `Bestimmung_k_Allgemein`

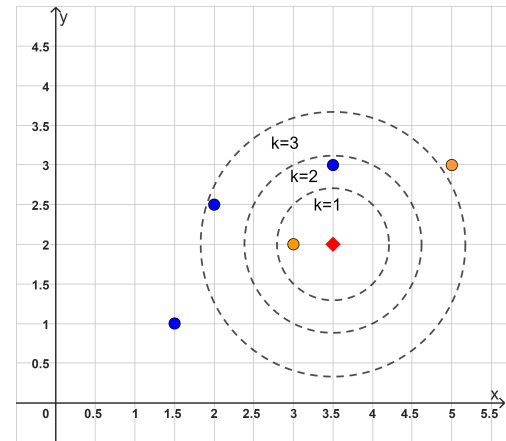


- **Validierungsdatenpunkt** (3.5 | 2):

Als Validierungsdatenpunkt P nehmen wir den ersten Trainingsdatenpunkt (3.5 | 2) mit dem erwarteten Label **blau** aus den Trainingsdaten heraus.

Es verbleiben somit folgende Trainingsdaten:

x	y	Label
5	3	orange
1.5	1	blau
3.5	3	blau
2	2.5	blau
3	2	orange



Mit den verbleibenden Trainingsdaten berechnen wir das Label für P für $k = 1, 2, 3$ und vergleichen das berechnete Label mit dem erwarteten Label **blau** des Validierungsdatenpunkts P .

Die Ergebnisse werden in der folgenden Tabelle dargestellt:

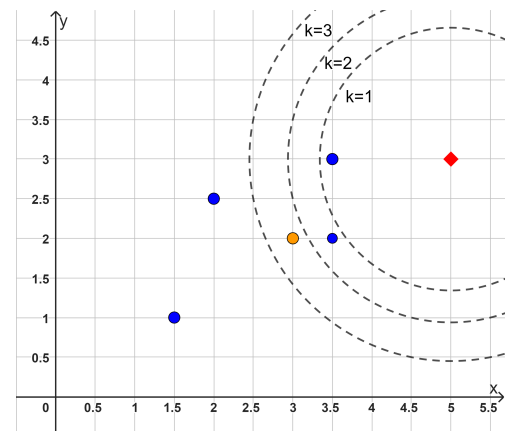
	$k = 1$	$k = 2$	$k = 3$	...
(3.5 2)	falsch	falsch	korrekt	...

- **Validierungsdatenpunkt** (5 | 3):

Als Validierungsdatenpunkt P nehmen wir den zweiten Trainingsdatenpunkt (5 | 3) mit dem erwarteten Label **orange** aus den Trainingsdaten heraus.

Es verbleiben somit die folgenden Trainingsdaten:

x	y	Label
3.5	2	blau
1.5	1	blau
3.5	3	blau
2	2.5	blau
3	2	orange



Mit den verbleibenden Trainingsdaten berechnen wir das Label für P für $k = 1, 2, 3$ und vergleichen das berechnete Label mit dem erwarteten Label **orange** des Validierungsdatenpunkts P .

Die Ergebnisse werden der folgenden Tabelle dargestellt:

	$k = 1$	$k = 2$	$k = 3$	...
(3.5 2)	falsch	falsch	korrekt	...
(5 3)	falsch	falsch	falsch	...



Gegebenenfalls sind Anpassungen der Trainingsdaten, des Hyperparameters sowie weitere Trainingsphasen notwendig.

Vorgehen:

- (a) Berechnung der Label für die Testdaten unter Verwendung des trainierten KNN-Modells
- (b) Vergleich der berechneten Label mit dem jeweils zugehörigen erwarteten Label

Die Ergebnisse der einzelnen Tests können beispielsweise in Form einer **Konfusionsmatrix** übersichtlich dargestellt werden.

Beispiel: Unausgefüllte Konfusionsmatrix für den Schulshirt-Kontext

	Berechnetes Label: S	Berechnetes Label: M	Berechnetes Label: L
Erwartetes Label: S			
Erwartetes Label: M			
Erwartetes Label: L			

Eine Größe, welche uns ein Maß für die Güte des trainierten Modells liefert, ist die **Genauigkeit**. Diese bestimmt sich als der Anteil der korrekt klassifizierten Testdatensätze an allen Testdatensätzen.

Beispiel:

Konfusionsmatrix:

	Berechnet: A	Berechnet: B
Erwartet: A	4	1
Erwartet: B	3	12

Genauigkeit:

$$\frac{4 + 12}{4 + 12 + 1 + 3} = \frac{16}{20} = 80\%$$

Arbeitsauftrag 11: Testen des KNN-Modells

- Öffnen Sie die Excel-Mappe `k_naechster_Nachbar_Klassifikation_Datensatz_T_Shirt` und navigieren Sie zur Tabelle `Testen_Konfusionsmatrix`
- Ergänzen Sie auf Basis der restlichen Trainingsdaten und $k = 4$ die Konfusionsmatrix
- Welche Genauigkeit ergibt sich für die gegebenen Testdaten?

Testdaten:			Aktuelle Testdatensatz		
Körpergröße	Brustumfang	Erwartetes Label	Körpergröße	Brustumfang	Berechnetes Label
174	90	S	174	90	S
170	88	S	170	88	
176	100	S	176	100	
181	106	M	181	106	
192	123	L	192	123	L

Parameter k:	4
--------------	---

Hinweis: Unterstützt nur k bis 10

Konfusionsmatrix:

Erhöhen Sie jeweils den Eintrag der Tabelle um 1 in der Zeile, welche sich aus dem obigen **erwarteten Label** und dem obigen **berechneten Label** zusammensetzt.

	Berechnet: S	Berechnet: M	Berechnet: L
Erwartet: S	0	0	0
Erwartet: M	0	0	0
Erwartet: L	0	0	0

Gesamtheit:	8	8	8
-------------	---	---	---

#DIV/0!



2.5 Mögliche Einflussfaktoren

Im Hinblick auf den k -nächste-Nachbarn-Algorithmus werden nun einige Einflussfaktoren betrachtet, welche sich problematisch auf die beabsichtigte Funktionsweise des Algorithmus auswirken können.

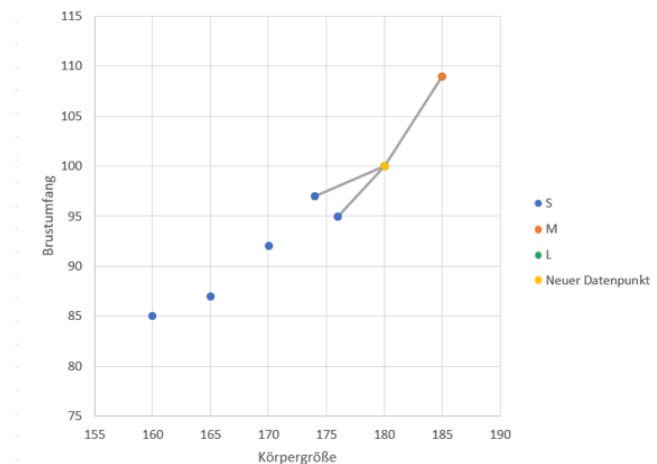
a) Ungleichmäßige Verteilung der Daten auf die betrachteten Klassen:

Bei der Klassifikation von Datenpunkten mit Hilfe des KNN-Algorithmus kann eine ungleichmäßige Verteilung der Trainingsdaten zu einer Verzerrung der Ergebnisse führen. Gehört ein unverhältnismäßig großer Teil der Trainingsdaten zur selben Klasse, beeinflusst dies die Prognose der Klasse eines neuen Datensatzes, da bei der Bestimmung der k nächsten Nachbarn mit hoher Wahrscheinlichkeit Datenpunkte der dominierenden Klasse unter den Nachbarn des neuen Datenpunkts sind, egal ob dieser zur dominierenden Klasse gehört oder nicht.

Beispiel Schulshirts:

Betrachten wir den in der Tabelle dargestellten Datensatz aus unserem Schulshirt-Kontext. Da 5 der 6 Datensätze zur Klasse Größe S gehören, wird für $k \geq 3$ ($k \geq 2$, falls man sich bei Gleichstand unter den Nachbarn auch für Größe S entscheidet) ein neuer Datenpunkt sicher der Klasse S zugeordnet, egal welche Eingabewerte der Datenpunkt hat.

Körpergröße	Brustumfang	Größe
165	87	S
174	97	S
176	95	S
160	85	S
170	92	S
185	109	M



b) Einseitigkeit der Daten (u.a. Semantik):

Bei der Klassifikation von Datenpunkten mit Hilfe des KNN-Algorithmus kann eine Einseitigkeit der Daten zu falschen Ergebnissen führen.

Beispiel "Was bin ich? – Stadt, Land, Tier-Edition":

Besteht der Teil der Trainingsdaten für die Klasse *Stadt* nur aus den Namen deutscher Städte (z.B. *Berlin*, *Hamburg*, *München*, *Frankfurt*, *Dresden*, *Köln*, *Bremen*, *Düsseldorf*, *Leipzig*, *Passau*), treten Probleme bei der Klassifikation internationaler Städte auf. So führt beispielsweise die Eingabe von *New York* bzw. *Washington* zur Fehlklassifikation *Land*.

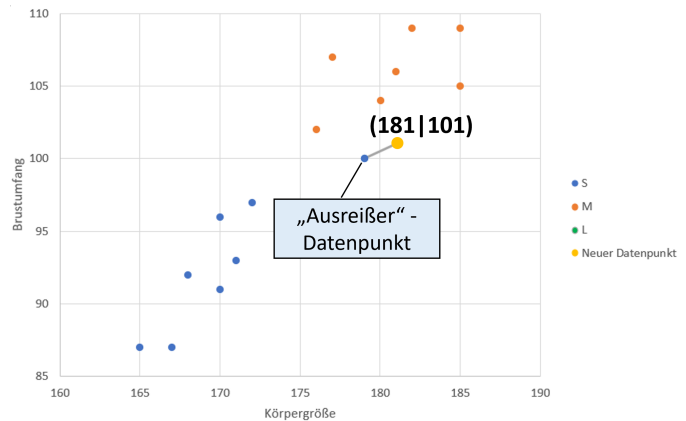
c) Wahl eines zu großen oder zu kleinen Werts für k :

Wählt man den Hyperparameter k sehr klein bzw. sehr groß, kann es sein, dass „Ausreißer“ in den Datenpunkten zu einer Fehlklassifikation führen bzw. dass eine große Anzahl an ausgewählten Nachbarn die Klarheit des Ergebnisses reduzieren.



Beispiel – Probleme durch einen zu kleinen Wert für k :

Sei $k = 1$. Wir wollen den neuen Datenpunkt (181 | 101) klassifizieren. Aufgrund seiner Lage würden wir für diesen Punkt die Klassifikation M erwarten. Allerdings liegt der Trainingsdatenpunkt (179 | 100) (Ausreißer-Datenpunkt) der Klasse S als 1-nächster Nachbar dem neuen Datenpunkt am nächsten, weshalb dieser fälschlicherweise der Klasse S statt der Klasse M zugeordnet wird. Eine Erhöhung von k (z.B. $k = 3$) würde wieder zu einer korrekten Klassifikation führen.

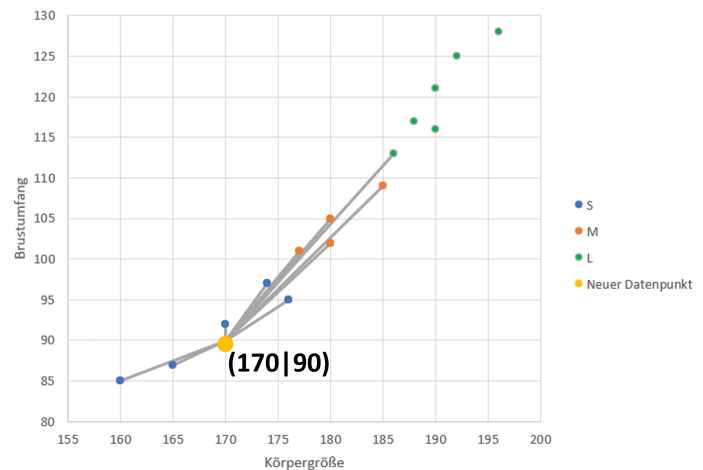


Beispiel – Probleme durch einen zu großen Wert für k :

Sei $k = 10$. Wir wollen den neuen Datenpunkt (170 | 90) klassifizieren. Die 10 nächsten Nachbarn gehören zu den folgenden Klassen:

$$5 \times S, 4 \times M, 1 \times L$$

Dies führt zu einer knappen Entscheidung für Größe S. Wäre beispielsweise $k=5$ gewesen, hätte dies eine eindeutigere Entscheidung für Größe S geliefert.



d) Notwendige Normierung von Daten:

Liegen die Merkmale der Trainingsdaten in unterschiedlichen Wertebereichen vor, kann dies bei der Abstandsberechnung zu Problemen führen.

Beispiel Lebensmittel(100g) – Merkmale Zuckermenge(in g) und Anteil am täglichen Kalorienbedarf(2000kcal):

- Das Merkmal Zuckermenge liegt in Gramm vor und kann Werte im Bereich $[0; 100]$ annehmen. Die maximal mögliche Differenz (Abstand in dieser Komponente) beträgt somit 100.
- Das Merkmal Anteil am täglichen Kalorienbedarf liegt als Dezimalzahl vor und kann Werte im Bereich $[0; 1]$ annehmen. Die maximal mögliche Differenz (Abstand in dieser Komponente) beträgt somit 1.

Die Vergleichbarkeit dieser beiden Merkmale ist also nur bedingt gegeben, da die Differenz des Merkmals Zuckermenge in den meisten Fällen einen höheren Einfluss auf die Bestimmung der am nächsten liegenden Nachbarn haben wird. Eine mögliche Lösung stellt die **Normierung**, also die Umrechnung der Datenwerte in den Bereich $[0; 1]$ dar. Dadurch können die Merkmale vergleichbar gemacht werden.

Fortsetzung Beispiel - Normierung des Merkmals Zuckermenge:

Betrachten wir die Zuckermenge 75g im Wertebereich $[0; 100]$. Dieser kann folgendermaßen in den Bereich $[0; 1]$ umgerechnet werden:



$$\frac{75 - 0}{100 - 0} = \frac{75}{100} = 0.75$$

Normiert entspricht eine Zuckermenge von 75g also dem Wert 0.75.

2.6 Regression als weitere Anwendung des KNN-Algorithmus

Nachdem wir bisher den k -nächste-Nachbarn-Algorithmus zur Klassifikation von Datenpunkten verwendet haben, beschäftigen wir uns nun mit einem weiteren Anwendungsbereich des Algorithmus, der Regression.

Überblick (Regression mit Hilfe des KNN-Algorithmus)

Eine **Regression** beschreibt den Zusammenhang zwischen zwei oder mehr Variablen. Dabei unterscheidet man unabhängige Variablen und abhängige Variablen (Zielgrößen). Mit der Regression können auf Basis der unabhängigen Variablen Prognosen über die abhängigen Variablen aufgestellt werden.

Vereinfachend betrachten wir im Folgenden nur den Fall einer einzelnen Zielgröße.

Vorgehen:

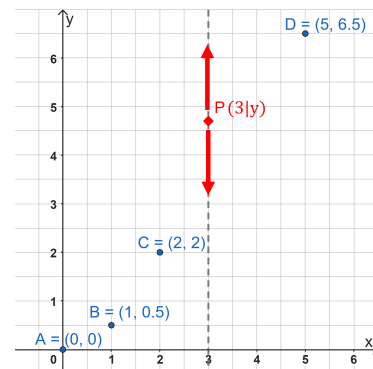
Wir wollen eine Prognose über die Zielgröße eines neuen Datenpunkts aufstellen von dem wir nur die Werte der unabhängigen Variablen kennen.

- Bilden des Mittelwerts der Zielgrößenwerte der k nächsten Nachbarn des neuen Datenpunkts
- Die Zielgröße des neuen Datenpunkts erhält als Wert den berechneten Mittelwert

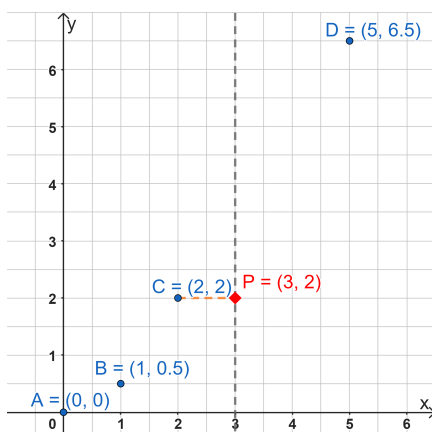
Beispiel:

Gegeben ist eine Menge an Trainingsdatenpunkten (blau). Die unabhängige Variable ist dabei die x -Koordinate, die abhängige Variable, d.h. die Zielgröße, die y -Koordinate.

Für den neuen Datenpunkt P soll nun für dessen x -Koordinate $x = 3$ die zugehörige y -Koordinate prognostiziert werden.



- $k = 1$:



Für $k = 1$ wird der bezüglich x (unabhängige Variable) nächste Nachbar (orange) gesucht. Dabei handelt es sich um $C(2 | 2)$.

Der Mittelwert der y -Koordinate (Zielgröße / abhängige Variable) der k nächsten Nachbarn ist hier, da es nur ein Datenpunkt ist, direkt die y -Koordinate des Punkts C .

Die prognostizierte y -Koordinate des neuen Datenpunkts P ist somit $y = 2$.



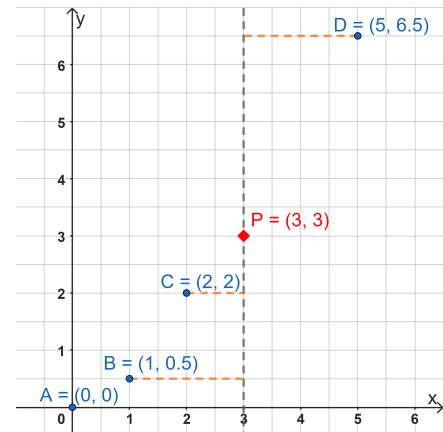
- $k = 3$:

Für $k = 3$ sind die bezüglich x nächsten Nachbarn (orange) die Punkte B , C und D .

Der Mittelwert der y -Koordinaten der k nächsten Nachbarn berechnet sich folgendermaßen:

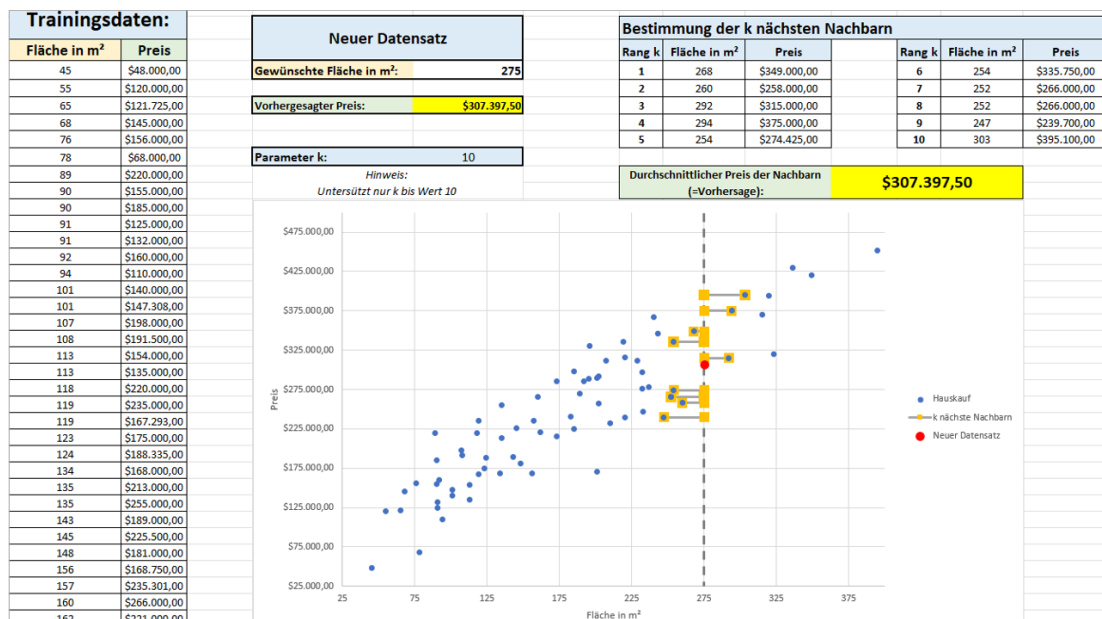
$$\frac{1}{3} \cdot (0.5 + 2 + 6.5) = \frac{1}{3} \cdot 9 = 3$$

Die prognostizierte y -Koordinate des neuen Datenpunkts P ist der berechnete Mittelwert, also $y = 3$.



Anwendung (Vorhersage von Hauspreisen)

Als mögliche Anwendung wurde eine Excel-Mappe zur Vorhersage von potentiellen Hauspreisen entwickelt, welche Regression unter Verwendung des KNN-Algorithmus nutzt um zu einer gewünschten Hausfläche(unabhängige Variable) den potentiellen Preis der Immobilie(Zielgröße/abhängige Variable) zu prognostizieren. Datengrundlage hierfür ist ein Auszug aus dem Datensatz der Sacramento Bee, welche Daten über Hauskäufe in Sacramento gesammelt und öffentlich zur Verfügung gestellt hat. Die Excel-Mappe nutzt nur einen Teildatensatz und verwendet nur die Merkmale *Hausfläche* und *Hauspreis*.



Arbeitsauftrag 12: Vorhersage von Hauspreisen

- Öffnen Sie die Excel-Mappe `k_naechster_Nachbar_Regression_Hauspreise` (siehe Anwendung)
- Probieren Sie die Tabelle für unterschiedliche Werte von k und Hausflächen Ihrer Wahl aus.

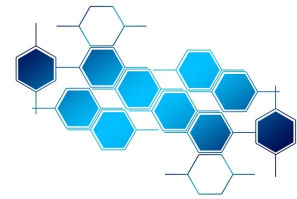


3. Künstliches Neuron – Das Perzeptron

3.1 Kompetenzerwartungen laut Lehrplan

Kompetenzerwartungen Informatik 11 (NTG / *spät beginnend*):

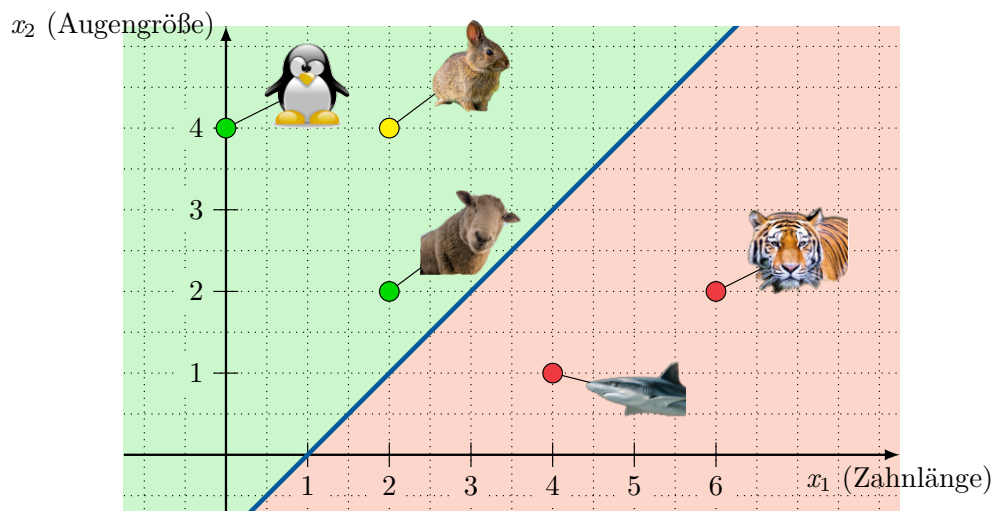
- erläutern die Funktionsweise eines künstlichen Neurons (Perzeptron) und beschreiben den grundsätzlichen Aufbau eines künstlichen neuronalen Netzes.
- implementieren / *simulieren* ein künstliches Neuron.



3.2 Aufbau und Funktionsweise eines Perzeptrons

Beispiel (Grundlegende Funktionsweise eines Perzeptrons)

Ein **Perzeptron** kann als linearer Klassifikator angesehen werden, der eine Punktmenge bezüglich einer Eigenschaft trennt. Als Beispiel könnte anhand der Augengröße und der Zahnlänge bei Tieren eingeschätzt werden, ob das Tier gefährlich oder ungefährlich⁴ ist. Anhand von gelabelten Trainingsdaten (hier grüne und rote Datenpunkte) „lernt“ das Perzeptron Parameter (Gewichte w_1, w_2 , Schwellenwert s). Diese Parameter legen eine Trenngerade (blau) fest, die die Ebene in zwei Hälften unterteilt. Die Parameter bzw. die dadurch festgelegte Trenngerade ermöglichen es dem Perzeptron neue Tiere als gefährlich oder ungefährlich einzuteilen (zu klassifizieren):

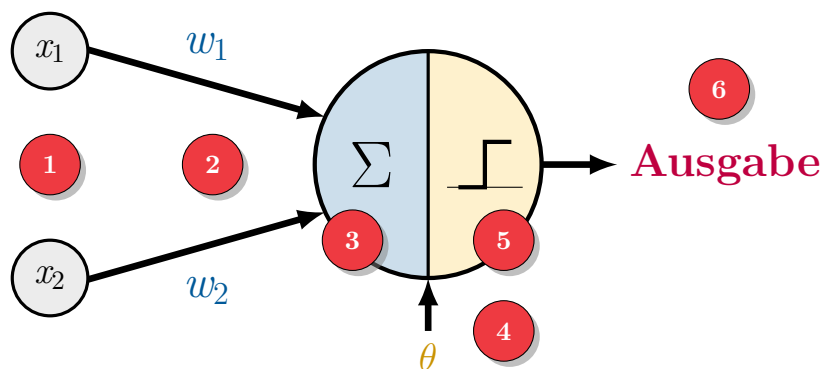


Möchte man nun ein neues Tier, von dem nicht bekannt ist, ob es gefährlich ist oder nicht (z.B. ein Hase (gelber Punkt) mit Zahnlänge $x_1 = 2$ und Augengröße $x_2 = 4$) klassifizieren, berechnet das Perzeptron mit Hilfe der Eingaben (x_1, x_2) und den gelernten Parametern die Ausgabe gefährlich oder ungefährlich. Grafisch wird dieses Ergebnis durch die Lage des Punktes hinsichtlich der Trenngeraden repräsentiert. Da der Punkt $(2, 4)$ oberhalb der Geraden liegt, wird das Tier als ungefährlich eingestuft.

⁴Die Idee, Tiere anhand ihrer Zahnlänge und Augengröße als gefährlich oder ungefährlich einzustufen, wurde von inf-schule.de (Überwachtes Lernen mit Neuronalen Netzen – Perzeptron als künstliches Neuron; (CC-BY-SA)) übernommen und verbleibt unter dieser Lizenz. Zuletzt aufgerufen am 17. Oktober 2022.



Überblick (Aufbau eines Perzeptrons)



Erläuterung der verschiedenen Bestandteile des Perzeptrons:

- 1 **Eingaben** x_1, x_2
- 2 **Gewichte** w_1, w_2
Mit den Gewichten kann den verschiedenen Eingaben eine unterschiedlich starke Bedeutung zugewiesen werden.
- 3 **Gewichtete Summe** $w_1 \cdot x_1 + w_2 \cdot x_2$
- 4 **Schwellenwert** θ für Aktivierungsfunktion
- 5 **Treppenfunktion** f (Aktivierungsfunktion)

$$f(a) = \begin{cases} 1 \text{ (ungefährlich)} & \text{falls } a \geq \theta \\ 0 \text{ (gefährlich)} & \text{falls } a < \theta \end{cases}$$
- 6 **Ausgabe:** 1 (ungefährlich) oder 0 (gefährlich)

Beispiel (Klassifikation durch ein Perzeptron)

Im einführenden Beispiel haben wir den Hasen ($x_1 = 2$ und $x_2 = 4$) anhand der Lage des Punktes als ungefährlich klassifiziert. Das Perzeptron muss diese Entscheidung allerdings durch Rechnung treffen.

Gegeben sind die Gewichte $w_1 = -1$ und $w_2 = 2$ sowie der Schwellenwert $\theta = 3$.

Gewichtete Summe:

$$a = w_1 \cdot x_1 + w_2 \cdot x_2 = -1 \cdot 2 + 2 \cdot 4 = 6$$

Auswertung der Treppenfunktion:

Auswertung von $f(a) = \begin{cases} 1 \text{ (ungefährlich)} & \text{falls } a \geq 3 \\ 0 \text{ (gefährlich)} & \text{falls } a < 3 \end{cases}$ mit Schwellenwert $\theta = 3$ für die gewichtete Summe $a = 6$ liefert 1 (ungefährlich).

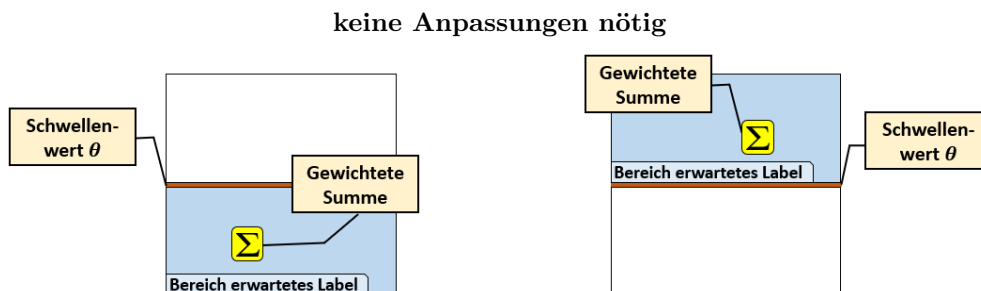


3.3 Wie lernt ein Perzeptron?

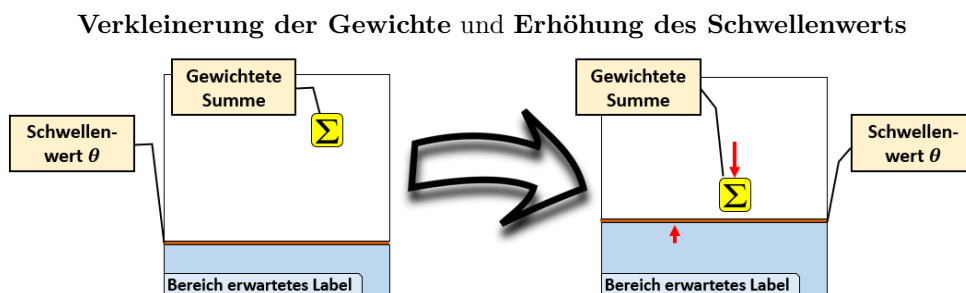
Überblick (Idee des Lernalgorithmus)

Es gibt drei Fälle zu unterscheiden:

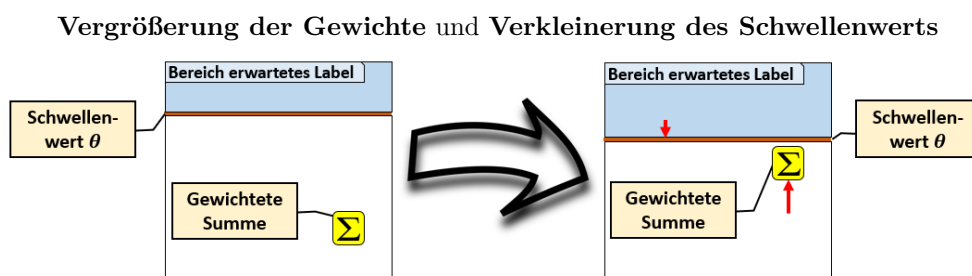
- (a) Berechnetes Ergebnis stimmt mit erwartetem Label überein, da die gewichtete Summe im Bereich des erwarteten Labels liegt:



- (b) Gewichtete Summe zu groß, da sie oberhalb des Schwellenwertes liegt:



- (c) Gewichtete Summe zu klein, da sie unterhalb des Schwellenwertes liegt:



Überblick (Lernalgorithmus Perzeptron (ohne Lernrate))

- (1) Wähle einen Trainingsdatensatz $[(x_1 \mid x_2); t]$
- (2) Berechne für $(x_1 \mid x_2)$ die gewichtete Summe $a = w_1 \cdot x_1 + w_2 \cdot x_2$ sowie die Ausgabe $f(a)$ des Perzeptrons
- (3) Berechne den Fehler $\delta = t - f(a)$



(4) Wenn $\delta = 1$:

$$w_1 := w_1 + x_1, \quad w_2 := w_2 + x_2, \quad \theta := \theta - 1$$

Wenn $\delta = -1$:

$$w_1 := w_1 - x_1, \quad w_2 := w_2 - x_2, \quad \theta := \theta + 1$$

Sonst ($\delta = 0$): Perzeptron hat Label korrekt berechnet

(5) Wiederhole Schritt (1)-(4) so lange, bis alle Punkte korrekt eingeordnet sind.

Arbeitsauftrag 13: Lernalgorithmus schrittweise durchführen

Gelabelte Trainingsdaten: (0 | 4), (2 | 2) (beide Label 1) (4 | 1), (6 | 2) (beide Label 0)

Vorbereitung: Wähle Gewichte w_1, w_2 und Schwellenwert θ zufällig: $w_1 = 1, w_2 = 1, \theta = 1$

(1) Erster Durchlauf:

(a) Erster Trainingsdatensatz: [(2 | 2); 1]

$(x_1 x_2)$	Label t	Gewicht w_1	Gewicht w_2	Schwellenwert θ	Gew. Summe a	$f(a)$
(2 2)	1	1	1	1	4	1

$$f(w_1 \cdot x_1 + w_2 \cdot x_2) = f(1 \cdot 2 + 1 \cdot 2) = f(4) = 1$$

$$\delta = t - f(4) = 1 - 1 = 0 \quad (\text{keine Anpassung nötig})$$

(b) Zweiter Trainingsdatensatz: [(4 | 1); 0]

$(x_1 x_2)$	Label t	Gewicht w_1	Gewicht w_2	Schwellenwert θ	Gew. Summe a	$f(a)$
(4 1)	0	1	1	1	5	1

$$f(w_1 \cdot x_1 + w_2 \cdot x_2) = f(1 \cdot 4 + 1 \cdot 1) = f(5) = 1$$

$$\delta = t - f(5) = 0 - 1 = -1 \quad (\text{Anpassung nötig})$$

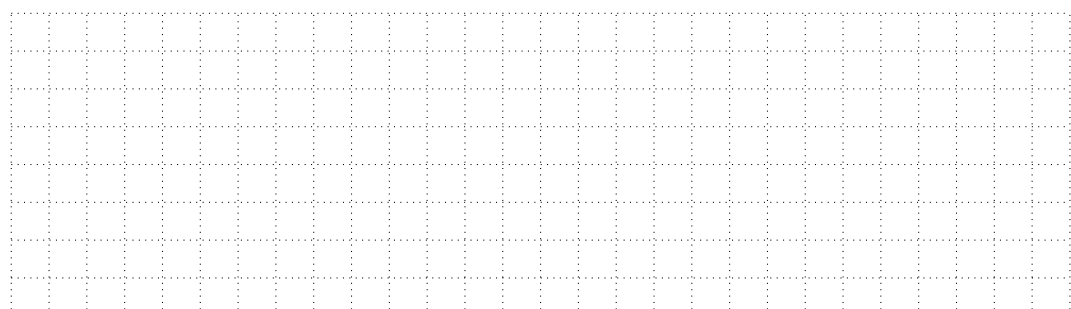
$$w_1 := 1 - 4 = -3$$

$$w_2 := 1 - 1 = 0$$

$$\theta := 1 + 1 = 2$$

(c) Dritter Trainingsdatensatz: [(6 | 2); 0]

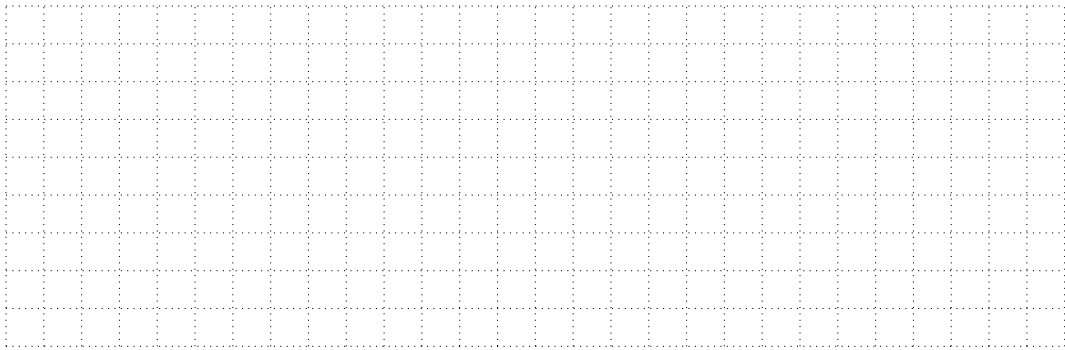
$(x_1 x_2)$	Label t	Gewicht w_1	Gewicht w_2	Schwellenwert θ	Gew. Summe a	$f(a)$



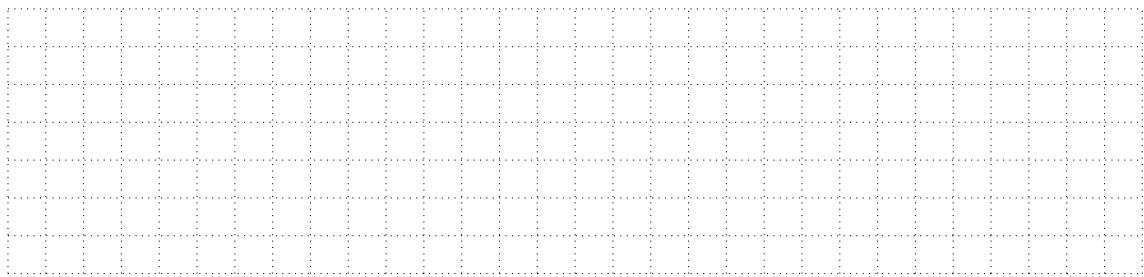


(d) Vierter Trainingsdatensatz: $[(0 \mid 4); 1]$

$(x_1 \mid x_2)$	Label t	Gewicht w_1	Gewicht w_2	Schwellenwert θ	Gew. Summe a	$f(a)$

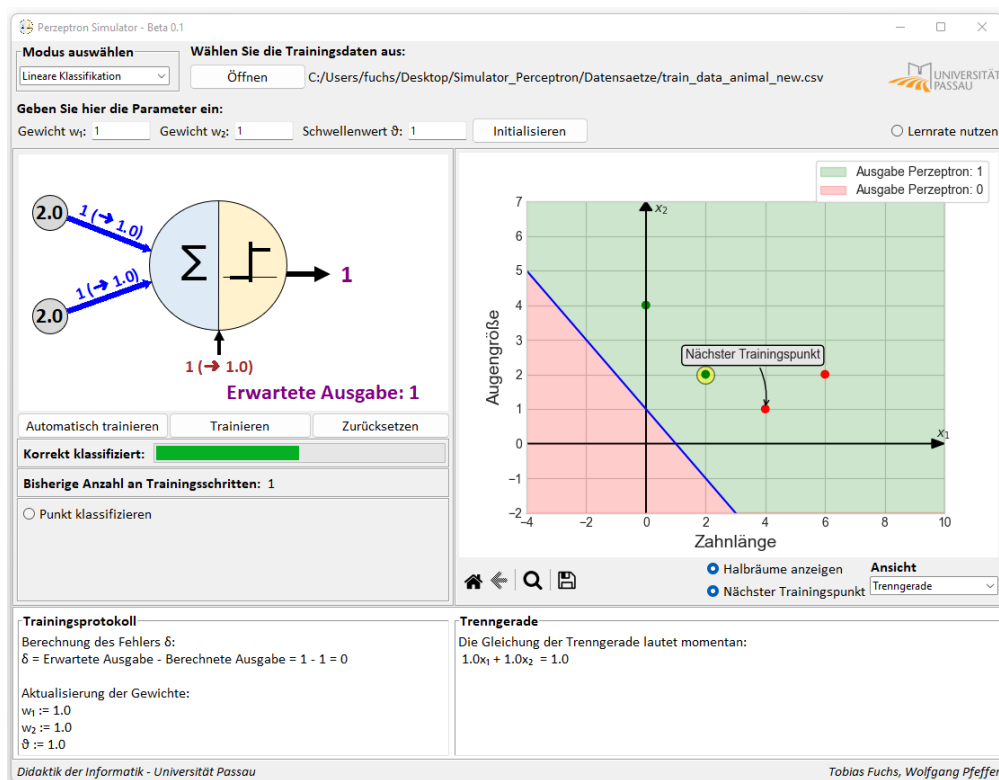


(2) **Für Schnelle:** Zweiter Durchlauf – Überprüfen Sie, ob alle Trainingsdatenpunkte korrekt klassifiziert werden, oder ob erneut Anpassungen der Gewichte erforderlich sind:



Arbeitsauftrag 14: Perzeptron-Lernprozess im Simulator

Zur Veranschaulichung und Simulation des Lernprozesses wurde ein **Perzeptron Simulator** entwickelt:





- Laden Sie den Ordner **Material Perzeptron** aus Mebis herunter.
- Öffnen Sie die Datei **Trainingsdaten_Tiere** im Simulator. Vollziehen Sie den durchgeführten Lernprozess schrittweise nach.
- Erweitern Sie den Datensatz **Trainingsdaten_Tiere** um das Guanoko $[(4, 3); 1]$ und trainieren Sie das Perzeptron erneut. Wie lautet die Trenngerade nun?

- (d) Öffnen Sie die Datei `Trainingsdaten_Zwei` im Simulator. **Initialisieren** Sie das Perzeptron mit den Gewichten $w_1 = -2$, $w_2 = 1$ und Schwellenwert $\theta = 2$. Führen Sie den Lernprozess durch und geben Sie die Anzahl der benötigten Lernschritte an. Welches Problem tritt auf?

- (e) Nutzen Sie nun die Option **Lernrate** und **Initialisieren** Sie das Perzeptron mit Lernrate $\alpha = 0.5$ (bzw. $\alpha = 0.25$). Führen Sie den Lernprozess erneut durch. Was stellen Sie fest?

Überblick (Die Lernrate α)

- **Eine** Gewichts Anpassung ist abhängig von den **einzelnen** Eingabedaten. Je nach Eingabe ergeben sich potentiell sehr große „Sprünge“ bei der Gewichts Anpassung.
- Die Gewichte müssen sich auf passende Werte für den **gesamten Datensatz** einpendeln.
- Die **Lernrate α** stellt einen moderierenden Faktor dar und schwächt beispielsweise große „Sprünge“ ab. Somit nimmt sie Einfluss auf die Geschwindigkeit des Lernens bzw. die Anzahl der benötigten Trainingsschritten.

- Anpassung der Formel:

$$w_i := w_i + \alpha \cdot x_i \quad \text{bzw.} \quad w_i := w_i - \alpha \cdot x_i \quad (i = 1, 2) \quad \text{und} \quad \theta := \theta - \alpha \cdot 1 \quad \text{bzw.} \quad \theta := \theta + \alpha \cdot 1$$

- Es gibt **keine** optimale Lernrate für **alle** Arten von neuronalen Netzen.
- Eine ungeeignete Wahl der Lernrate kann den Lernprozess auch verlangsamen bzw. sogar scheitern lassen.

**Überblick (Lernalgorithmus Perzeptron (mit Lernrate α))**

- (1) Wähle einen Trainingsdatensatz $[(x_1 \mid x_2); t]$
- (2) Berechne für $(x_1 \mid x_2)$ die gew. Summe $a = w_1 \cdot x_1 + w_2 \cdot x_2$ sowie die Ausgabe $f(a)$ des Perzeptrons
- (3) Berechne den Fehler $\delta = t - f(a)$

- (4) Wenn $\delta = 1$:

$$w_1 := w_1 + \alpha \cdot x_1, \quad w_2 := w_2 + \alpha \cdot x_2, \quad \theta := \theta - \alpha \cdot 1$$

Wenn $\delta = -1$:

$$w_1 := w_1 - \alpha \cdot x_1, \quad w_2 := w_2 - \alpha \cdot x_2, \quad \theta := \theta + \alpha \cdot 1$$

Sonst ($\delta = 0$): Perzeptron hat Label korrekt berechnet

- (5) Wiederhole Schritt (1)-(4) so lange, bis alle Punkte korrekt eingeordnet sind.

Überblick (Verkürzung des Lernalgorithmus durch Hinzunahme des Parameters δ)

Betrachtet man die drei Fälle unter Punkt (4) im Lernalgorithmus genauer, erkennt man, dass man diese durch Hinzunahme des Faktors δ zusammenfassen kann. Es ergibt sich somit folgende gekürzte Fassung:

- (4) Anpassung der Gewichte und des Schwellenwerts:

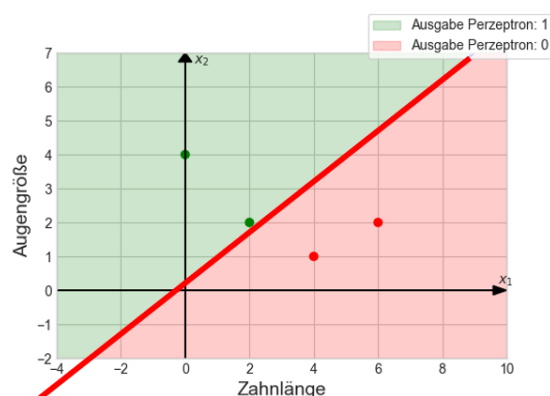
$$w_1 := w_1 + \delta \cdot \alpha \cdot x_1, \quad w_2 := w_2 + \delta \cdot \alpha \cdot x_2, \quad \theta := \theta - \delta \cdot \alpha \cdot 1$$

3.4 Geometrische Aspekte des Perzeptrons

Überblick (Trenngerade)

- Gewichte w_1 , w_2 und der Schwellenwert θ legen die Trenngerade fest
- Gleichung der Trenngeraden in Koordinatenform:

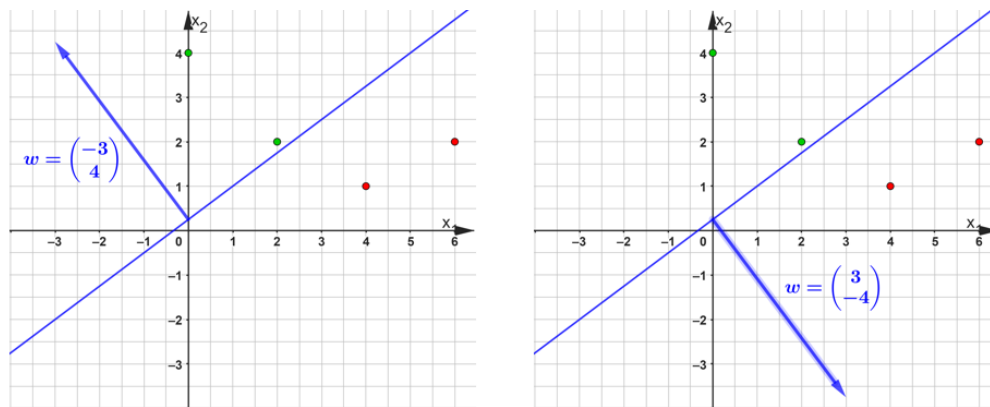
$$w_1 x_1 + w_2 x_2 = \theta$$

**Überblick (Gewichtsvektor als Normalenvektor der Gerade)**

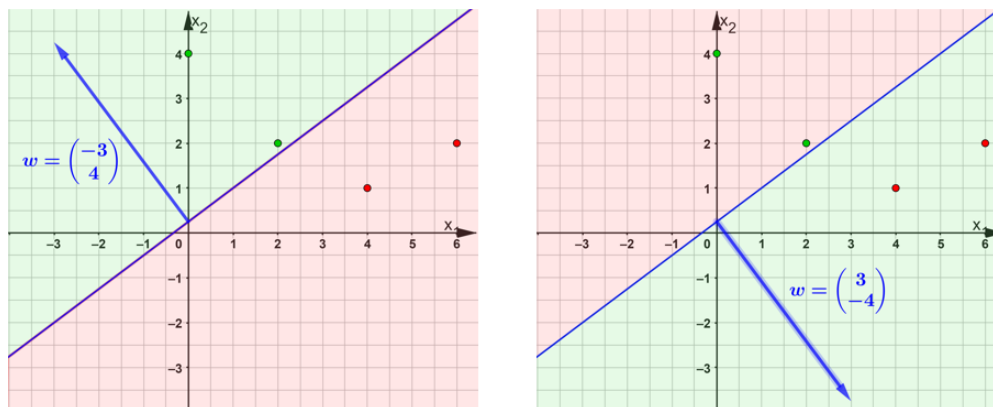
Wie wird festgelegt, welcher der beiden Halbräume bzgl. der Trenngerade / des Perzeptrons den Wert 1 bzw. den Wert 0 zugeordnet bekommt?



Beispielsweise beschreiben die zwei Variablenbelegungen $w_1 = -3, w_2 = 4, \theta = 1$ (linke Grafik) bzw. $w_1 = 3, w_2 = -4, \theta = -1$ (rechte Grafik) dieselbe Gerade und teilen somit die Trainingsdaten gemäß ihrer Label:



Vom Perzeptron werden dabei im ersten Fall (linke Grafik) vier von vier Datensätzen korrekt, im zweiten Fall (rechte Grafik) null von vier Datensätzen korrekt klassifiziert. Dies liegt daran, dass das Perzeptron für alle Punkte des Halbraums, in den der Normalenvektor der Trenngerade zeigt, den Wert 1 liefert. Bei der graph. Darstellung eines Perzeptrons in Form einer Trenngerade sollte also immer auch bedacht werden, welcher Halbraum zu welchem Ergebnis des Perzeptrons (bzw. der dadurch repräsentierten Klasse) gehört:



Exkurs (Gradientenabstieg)

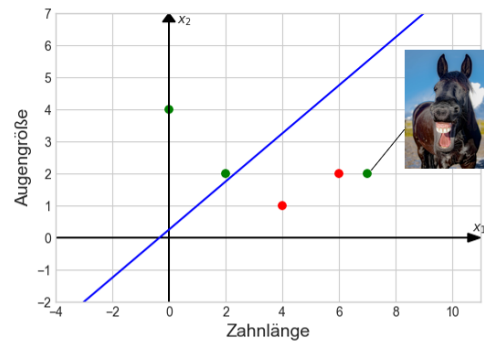
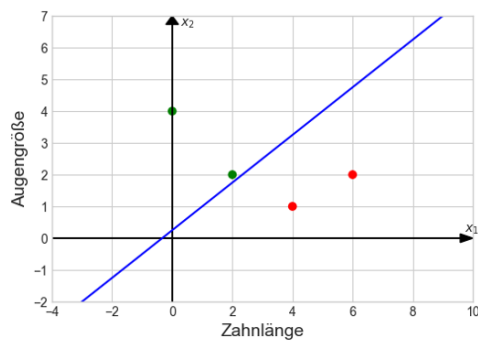


Wir haben gesehen, dass der Lernalgorithmus bei unseren Beispielen, die auch tatsächlich linear separierbar waren, funktioniert. Aber warum ist das so und welche zentrale mathematische Idee steckt dahinter? Bei Interesse können Sie sich hierzu unter dem [Buzzword Gradientenabstieg](#) informieren.

3.5 Grenzen des Perzeptrons

Überblick (Grenzen des Perzeptrons)

Damit der Lernalgorithmus des Perzeptrons geeignete Parameter und damit eine passende Trenngerade finden kann, müssen die Trainingsdaten gemäß ihrer Label linear separierbar sein:



Im linken Beispiel findet der Algorithmus eine passende Trenngerade, rechts ist dies nicht möglich

3.6 Implementierung des Perzeptrons

3.6.1 Implementierung in JAVA

Eine mögliche Implementierung des Perzeptrons in [JAVA](#) mit der Klasse `Datenpunkt` als Repräsentation eines Trainingsdatenpunktes:

Code 1: Perzeptron.java

```
/**
 * Die Klasse Perzeptron setzt die Funktionalitaet des Perzeptrons um
 */
public class Perzeptron {
    // Speicherung der Gewichte und des Schwellenwerts
    private double w_1;
    private double w_2;
    private double theta;
    // Speicherung der Lernrate
    private double lernrate = 1;

    /**
     * Konstruktor fuer Objekte der Klasse Perzeptron
     */
    public Perzeptron(double wert_w_1, double wert_w_2, double wert_theta, double wert_lernrate) {
        w_1 = wert_w_1;
        w_2 = wert_w_2;
        theta = wert_theta;
        lernrate = wert_lernrate;
    }

    /**
     * Trainiert das aktuelle Perzeptron mit dem Datenpunkt punkt
     */
    public void trainieren(Datenpunkt punkt) {
        // Berechnung der gewichteten Summe
        double gewichtete_Summe = w_1*punkt.gibEingabewertX1() + w_2*punkt.gibEingabewertX2();
    }
}
```



```
// Berechnung der Treppenfunktion
int berechnete_ausgabe = 0;
if (gewichtete_Summe >= theta) {
    berechnete_ausgabe = 1;
} else {
    berechnete_ausgabe = 0;
}

// Berechnung des Fehlers delta
int delta = punkt.gibLabel() - berechnete_ausgabe;

// Anpassung der Gewichte und des Schwellenwerts, falls noetig
if (delta == 1) {
    w_1 = w_1 + lernrate*punkt.gibEingabewertX1();
    w_2 = w_2 + lernrate*punkt.gibEingabewertX2();
    theta = theta - lernrate*1;
    System.out.println("Parameterwerte: w_1 = " + w_1 + "; w_2 = " + w_2 + "; theta = " +
        theta);
}
else if(delta == -1) {
    w_1 = w_1 - lernrate*punkt.gibEingabewertX1();
    w_2 = w_2 - lernrate*punkt.gibEingabewertX2();
    theta = theta + lernrate*1;
    System.out.println("Parameterwerte: w_1 = " + w_1 + "; w_2 = " + w_2 + "; theta = " +
        theta);
}
else {
    System.out.println("Keine Anpassung der Gewichte notwendig!");
}
}

/**
 * Klassifiziert den Punkt mit Eingabewerten x_1 und x_2 mit Hilfe des aktuellen Perzeptrons
 */
public int punktKlassifizieren(double x_1, double x_2) {
    // Berechnung der gewichteten Summe
    double gewichtete_Summe = w_1*x_1 + w_2*x_2;
    // Berechnung der Treppenfunktion
    int berechnete_ausgabe = 0;
    if (gewichtete_Summe >= theta){
        berechnete_ausgabe = 1;
    } else {
        berechnete_ausgabe = 0;
    }
    return berechnete_ausgabe;
}
```



```
}

/**
 * Gibt die Gleichung der momentanen Trenngerade auf der Konsole aus
 */
public void trenngeradeAusgeben(){
    System.out.println("Die Gleichung der Trenngerade lautet aktuell:");
    System.out.println(w_1 + "*x_1 + " + w_2 + "*x_2 = " + theta);
}

}
```

Code 2: Datenpunkt.java

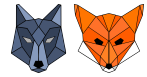
```
/**
 * Die Klasse Datenpunkt speichert einen Trainingsdatenpunkt des Perzeptrons
 */
public class Datenpunkt {
    //Speicherung der Eingabewerte
    private double x_1;
    private double x_2;
    // Speicherung des Labels
    private int label;

    public Datenpunkt(double wert_x_1, double wert_x_2, int wert_label) {
        x_1 = wert_x_1;
        x_2 = wert_x_2;
        label = wert_label;
    }

    public double gibEingabewertX1() {
        return x_1;
    }

    public double gibEingabewertX2() {
        return x_2;
    }

    public int gibLabel() {
        return label;
    }
}
```



3.6.2 Implementierung in Python

Eine mögliche Implementierung des Perzeptrons in der Programmiersprache Python:

Code 3: Perzeptron.py

```
import numpy

# Die Klasse Perzeptron setzt die Funktionalitaet des Perzeptrons um
class Perzeptron:

    # Konstruktor der Klasse Perzeptron
    def __init__(self, wert_w_1, wert_w_2, wert_schwellenwert, wert_lernrate=1):
        # Erstellt einen Vektor(2x1-Matrix) aus der Gewichtsliste [wert_w_1, wert_w_2]
        self.gewichtsvektor = numpy.array([wert_w_1, wert_w_2], ndmin=2).T
        self.schwellenwert = wert_schwellenwert
        self.lernrate = wert_lernrate

    # Trainiert das aktuelle Perzeptron mit dem Datenpunkt bestehend aus den Eingaben x_1 und x_2 und dem
    # Label label
    def trainieren(self, x_1, x_2, label):
        # Erstellt einen Vektor(2x1-Matrix) aus der Eingabenliste [x_1, x_2]
        eingabenvektor = numpy.array([x_1, x_2], ndmin=2).T
        # Berechnet die gewichtete Summe unter Verwendung der Matrixmultiplikation(dot) des Pakets numpy.
        # Der Gewichtsvektor wird mit T transponiert um die Matrixmultiplikation anwenden zu koennen
        gewichtete_summe = numpy.dot(self.gewichtsvektor.T, eingabenvektor)
        # Berechnung der Ausgabe des Perzeptrons durch Anwendung der Aktivierungsfunktion
        berechnete_ausgabe = self.aktivierungsfunktion(gewichtete_summe)
        # Berechnung des Fehlers delta
        delta = label - berechnete_ausgabe
        # Anpassung der Gewichte unter Verwendung von delta um die Fallunterscheidung zu vermeiden.
        # Python rechnet hier automatisch mit den Vektoren/Matrizen und wendet die Operationen
        # komponentenweise an
        self.gewichtsvektor = self.gewichtsvektor + delta * self.lernrate * eingabenvektor
        # Anpassung des Schwellenwerts unter Verwendung von delta, um die Fallunterscheidung zu vermeiden
        self.schwellenwert = self.schwellenwert - delta * self.lernrate * 1

    # Klassifiziert den Punkt mit Eingabewerten x_1 und x_2 mithilfe des aktuellen Perzeptrons
    def punktKlassifizieren(self, x_1, x_2):
        # Erstellt einen Vektor(2x1-Matrix) aus der Eingabenliste [x_1, x_2]
        eingabenvektor = numpy.array([x_1, x_2], ndmin=2).T
        # Berechnet die gewichtete Summe unter Verwendung der Matrixmultiplikation(dot) des Pakets numpy.
        # Der Gewichtsvektor wird mit T transponiert um die Matrixmultiplikation anwenden zu koennen
        gewichtete_summe = numpy.dot(self.gewichtsvektor.T, eingabenvektor)
        # Berechnung der Ausgabe des Perzeptrons durch Anwendung der Aktivierungsfunktion
        berechnete_ausgabe = self.aktivierungsfunktion(gewichtete_summe)
        return berechnete_ausgabe
```




```
# Aktivierungsfunktion des Perzeptrons
def aktivierungsfunktion(self, wert_gewichtete_summe):
    if wert_gewichtete_summe >= self.schwellenwert:
        return 1
    else:
        return 0

def ausgabeAktuelleParameter(self):
    print("Parameterwerte: w_1 = " + str(self.gewichtsvektor[0][0]) + "; w_2 = " + str(
        self.gewichtsvektor[1][0]) + "; schwellenwert = " + str(self.schwellenwert))

# Trainiert das Perzeptrons fuer unsere 4 Datenpunkte und gibt nach jedem Schritt die Parameter aus
if __name__ == '__main__':
    p = Perzeptron(1, 1, 1)
    p.trainieren(2, 2, 1)
    p.ausgabeAktuelleParameter()
    p.trainieren(4, 1, 0)
    p.ausgabeAktuelleParameter()
    p.trainieren(6, 2, 0)
    p.ausgabeAktuelleParameter()
    p.trainieren(0, 4, 1)
    p.ausgabeAktuelleParameter()
```

3.7 Testen des Modells

Überblick (Testen des Modells)

Wie bereits beim Entscheidungsbaum als auch beim k -nächste-Nachbarn-Algorithmus sollte das mit dem Perzeptron-Lernalgorithmus erstellte Modell wieder anhand von Testdaten auf seine Güte hin untersucht werden. Auch hier bietet sich eine Darstellung der richtig bzw. falsch klassifizierten Testdaten in einer Konfusionsmatrix an:

	Berechnet: Ungefährlich	Berechnet: Gefährlich
Erwartetes Label: Ungefährlich	2	1
Erwartetes Label: Gefährlich	0	1



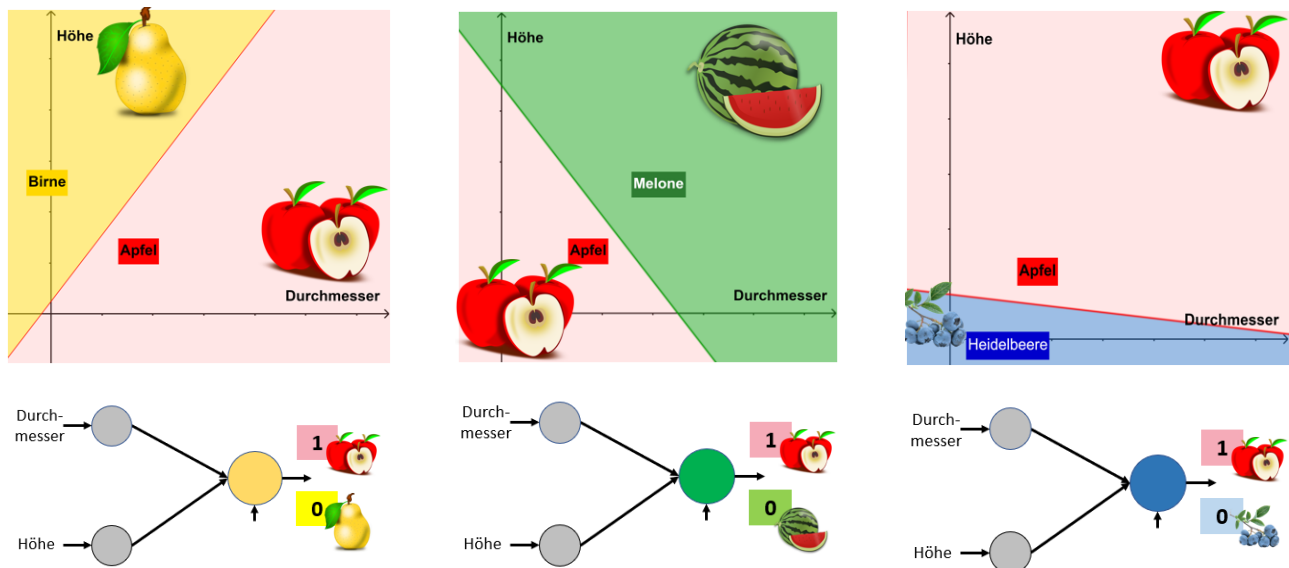
Die Genauigkeit des Modells beträgt demnach:

$$\frac{2+1}{2+1+1} = \frac{3}{4} = 75\%$$



3.8 Vom Perzeptron zum neuronalen Netz

Wir haben gesehen, dass ein Perzeptron als linearer Klassifikator genutzt werden kann. Beispielsweise können wir damit hinsichtlich der beiden Merkmale Durchmesser und Höhe jeweils einen Apfel von einer Birne, von einer Melone sowie von einer Heidelbeere unterscheiden:



Das Problem dabei ist, dass man hierbei verschiedene Perzeptrone erhält, die einen Apfel immer nur von einer zweiten Frucht unterscheiden können. Aber wie findet man nun die Äpfel unter allen vier Fruchtarten?

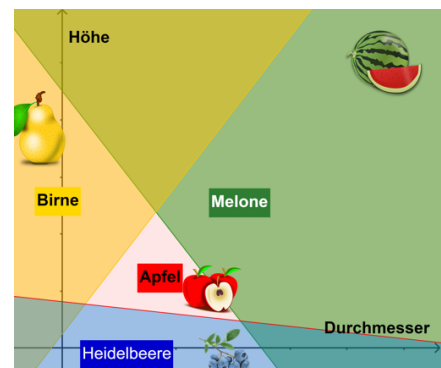
Arbeitsauftrag 15: Gemeinsam sind wir stark

Graphisch löst man das gegebene Problem, indem man alle drei Bilder mit ausreichend Transparenz übereinanderlegt.

- (a) Skizzieren Sie mit Hilfe der drei bekannten Perzeptrone nun einen **gemeinsamen** Apfelklassifikator.

Hinweis: Sie dürfen auch weitere Neuronen hinzufügen.

- (b) Tragen Sie an geeigneten Stellen bereits Gewichte bzw. einen Schwellenwert ein.



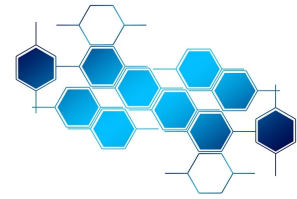
4. Chancen und Risiken von Künstlicher Intelligenz für Individuum und Gesellschaft

„KI ist wahrscheinlich das Beste oder das Schlimmste, was der Menschheit passieren kann“

Stephen Hawking

Kompetenzerwartungen Informatik 11 (NTG und spät beginnend):

- nehmen zu ausgewählten aktuellen Einsatzmöglichkeiten der Künstlichen Intelligenz Stellung und bewerten Chancen und Risiken für Individuum und Gesellschaft



4.1 Chancen und Risiken und deren Bedeutung⁵

Arbeitsauftrag 16: Chancen und Risiken

Formulieren Sie für sich vorab jeweils drei Chancen und Risiken, die durch den Einsatz von Künstlicher Intelligenz resultieren. Entscheiden Sie zudem, ob diese eher in Bezug auf das Individuum oder die gesamte Gesellschaft relevant sind:

Arbeitsauftrag 17: OpenAI – chatGPT

Rufen Sie die Seite <https://chat.openai.com/chat> auf und experimentieren Sie mit der künstlichen Intelligenz. Falls Sie keinen OpenAI-Account erstellen möchten, können Sie stattdessen auf der Seite <https://quickdraw.withgoogle.com> mit Google Quickdraw experimentieren.

Überblick (Fokussierung auf spezielle Entscheidungssysteme)

Fokussierung vor allem auf ethisch relevante algorithmische Entscheidungssysteme, die

- (a) über Menschen entscheiden.
- (b) über Ressourcen entscheiden, die Menschen betreffen.
- (c) Entscheidungen treffen, die gesellschaftliche Teilhabemöglichkeiten von Personen ändern.

⁵Die Inhalte und Beispiele dieses Abschnitts basieren auf dem Buch „Ein Algorithmus hat kein Taktgefühl“ von Prof. Dr. Katharina Zweig, welches wir nur wärmstens als Lektüre weiterempfehlen können.

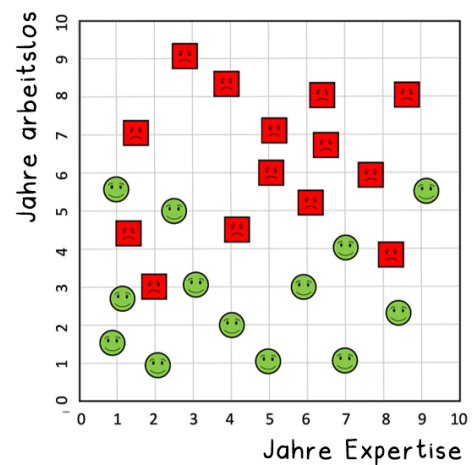


4.2 Die Wahl des Qualitätsmaßes und dessen Auswirkungen

Arbeitsauftrag 18: Qualitätsmaße

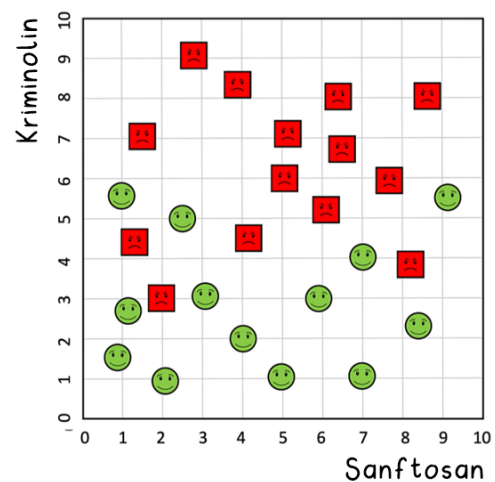
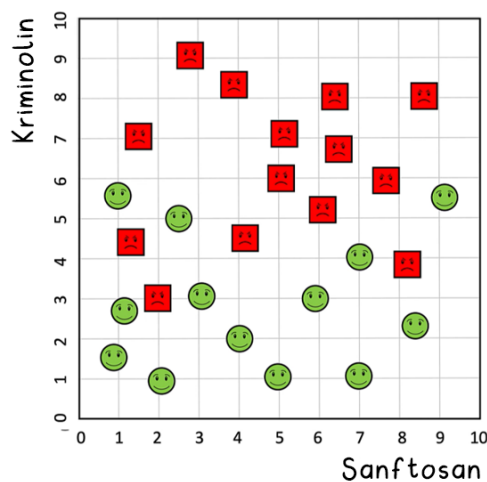
Sie sollen entscheiden, ob eine arbeitssuchende Person zum Vorstellungsgespräch eingeladen werden soll oder nicht.

- (a) Malen Sie hierzu zwischen die quadratischen (weniger erfolgreiche Arbeitnehmer:innen) und kreisförmigen (erfolgreiche Arbeitnehmer:innen) Smileys eine gerade Linie, so dass die Linie möglichst gut die quadratischen von den kreisförmigen Smileys trennt.



- (b) Es gibt eine neue Bewerbung einer Person mit 5,5 Jahren Berufserfahrung und insgesamt vier Jahren ohne Beschäftigung. Laden Sie diese Person ein oder nicht?

- (c) Wir betrachten jetzt nicht mehr arbeitssuchende Personen, sondern kriminelle und unschuldige Bürger (links) bzw. Terroristen und unschuldige Bürger (rechts). Zeichnen Sie erneut eine Trennlinie ein, mit der entschieden werden soll, ob eine Person als solche eingestuft werden soll oder nicht:



- (d) Es liegt hierbei immer eine gewisse Willkür bei der Wahl des Qualitätsmaßes vor. Als Beispiel hierfür dient der Ansatz von William Blackstone (*It is better that ten guilty persons escape than that one innocent suffer*) im Gegensatz zu der Ansicht von Dick Cheney (*I am more concerned with bad guys who got out and released than I am with a few that, in fact, were innocent*). Wie hätten diese beiden Herren die Linie eingezeichnet?

Beispiele und Grafiken entnommen aus „Ein Algorithmus hat kein Taktgefühl“ von Prof. Dr. Katharina Zweig



- **Diskriminierung durch fehlende Daten**

Bsp.: Spracherkennung von Schotten im Lift

<https://www.youtube.com/watch?v=MNuFcIRlwdc>

- **Diskriminierung durch Weglassen sensibler Informationen**

Es kann Benachteiligung entstehen, wenn dem Algorithmus sensitive Eigenschaften vorenthalten werden, diese aber ursächlich für unterschiedliches Verhalten sind.

Bsp.: Einstufung als Verbrecher unabhängig vom Geschlecht. Möglicherweise lässt sich durch Aufteilen der Datenmenge jeweils eine „optimale“ Gerade finden.

- **Diskriminierung durch dynamisches Weiterlernen**

Bsp.: Chatbot Tay

<https://dailywireless.org/internet/what-happened-to-microsoft-tay-ai-chatbot/>

(c) **Nachvollziehbarkeit von Entscheidungen**

Gefahr, dass KI-Systeme als Blackbox ungetestet zur Entscheidungsfindung eingesetzt werden. „Was ich aber nicht nachvollziehen kann, ist, dass eine Maschine etwas dürfen soll, was wir in jedem Labor der Welt für unwissenschaftlich halten: aus Beobachtungen Hypothesen zu entwickeln und diese ungetestet zur Beurteilung weiterer Situationen zu verwenden“ (Prof. Dr. Zweig). Gerade in Bezug auf Menschen wird gefordert, dass nachvollziehbar sein sollte, warum eine Personen einen Kredit nicht bekommen hat bzw. als rückfällig eingestuft wurde, etc.

(d) **Wer trägt die Verantwortung?**

Wer ist für ein KI-System verantwortlich? Der Entwickler? Der Datenlieferant? Der Nutzer?

Bsp: Autonomes Fahren

<https://www.moralmachine.net/>





(e) Schadenspotential

Bei der Beurteilung des möglichen Schadenspotentials kann unterschieden werden zwischen:

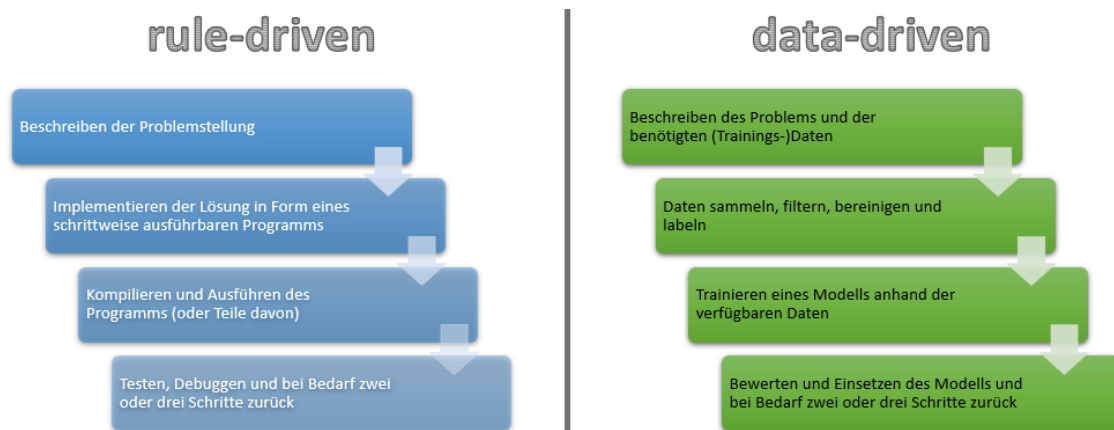
- Systemen, die die gesellschaftliche Teilhabe einzelner stark einschränken können (z.B. Identifizierung von Terroristen) (Harcourt, 2006)
- Systemen, die Öffentlichkeit erzeugen und auf dieser Ebene schaden können (z.B. algorithmische Entscheidungssysteme in sozialen Netzwerken) (Lischka & Klingel, 2017)

Zusammenfassung (Chancen und Risiken von KI-Systemen)

KI-Systeme bieten eine Fülle von Möglichkeiten, die uns im Alltag oder der Forschung unterstützen können, wie z.B. Bilderkennung, Text-to-Speech oder auch die chat-GPT. Man muss die Systeme aber vor allem dann kritisch beleuchten, wenn sie Entscheidungen über Menschen oder über Ressourcen, die Menschen betreffen, vornehmen (z.B. Einstellung zu Bewerbungsgesprächen, COMPAS-System in den USA, Bewertungssystem in China). Obige Mindmap zeigt eine Reihe von Aspekten, die bei der Beurteilung von KI-Systemen herangezogen werden können. Problematisch ist vor allem die Auswahl des Qualitätsmaßes – was einige als gerechten Ausgleich empfinden, wird von anderen als unfaire Diskriminierung aufgefasst. „Ideologiefreiheit wird an diesem strategischen Punkt weder in die eine noch in die andere Richtung möglich sein. Denn hier geht es ganz fundamental um die Frage, wie man Diskriminierung operationalisiert, wie man also das Ausmaß an Diskriminierung mit Zahlen bewertet. In einem zweiten Schritt muss dann bewertet werden, ob die festgestellte Diskriminierung ungerechtfertigt war oder nicht. Und was der eine als ideologiefrei empfindet, sieht der andere als ideologisch begründet und umgekehrt.“ (Prof. Dr. Zweig).

5. Paradigmenwechsel im Problemlöseprozess mit Algorithmen

Überblick (Paradigmenwechsel)



nach Vortrag von Matti Tedre auf der WiPSCE'22-Konferenz zu Computational Thinking 2.0